



UNIVERSITY OF TARTU

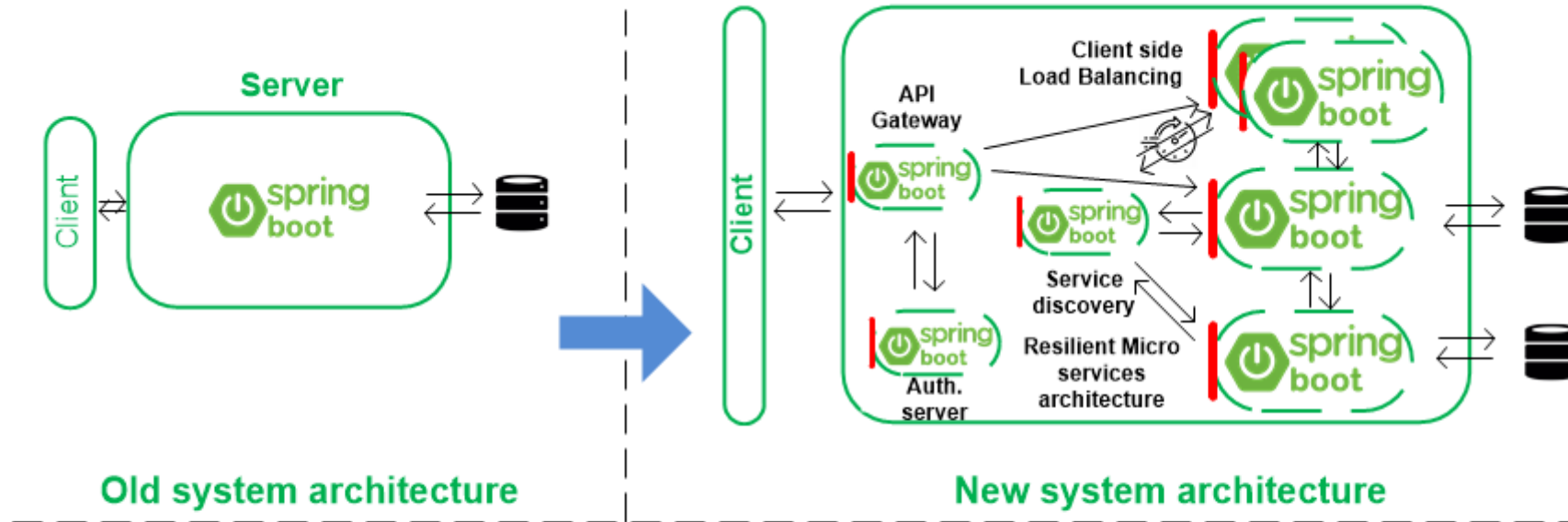


Enterprise System Integration (MTAT.03.229)

LECTURE 7: INTEGRATION PATTERNS IN MICROSERVICE ARCHITECTURE

**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

What we have learned?



How we can transfer the system from old to the new architecture?

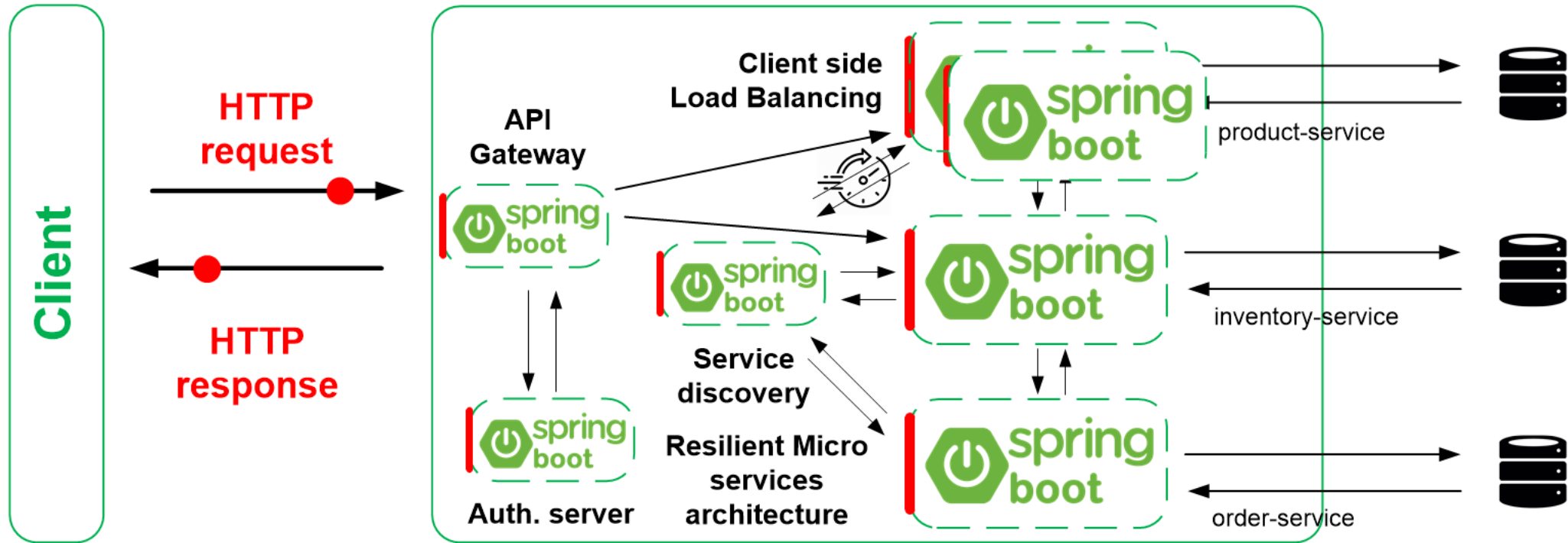
- Design patterns
- Implementation technologies



What we have learned?

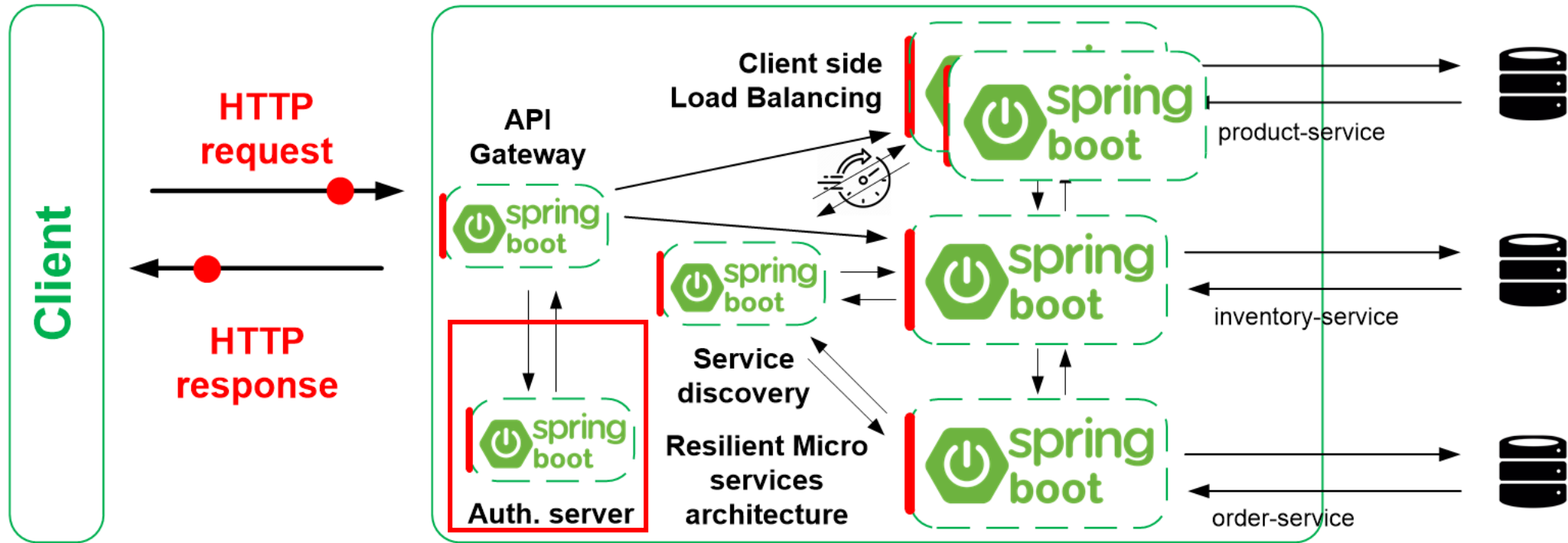
Implementation technologies

What we have learned?



Implementation technologies we have already learned ...
What we still need to learn ...

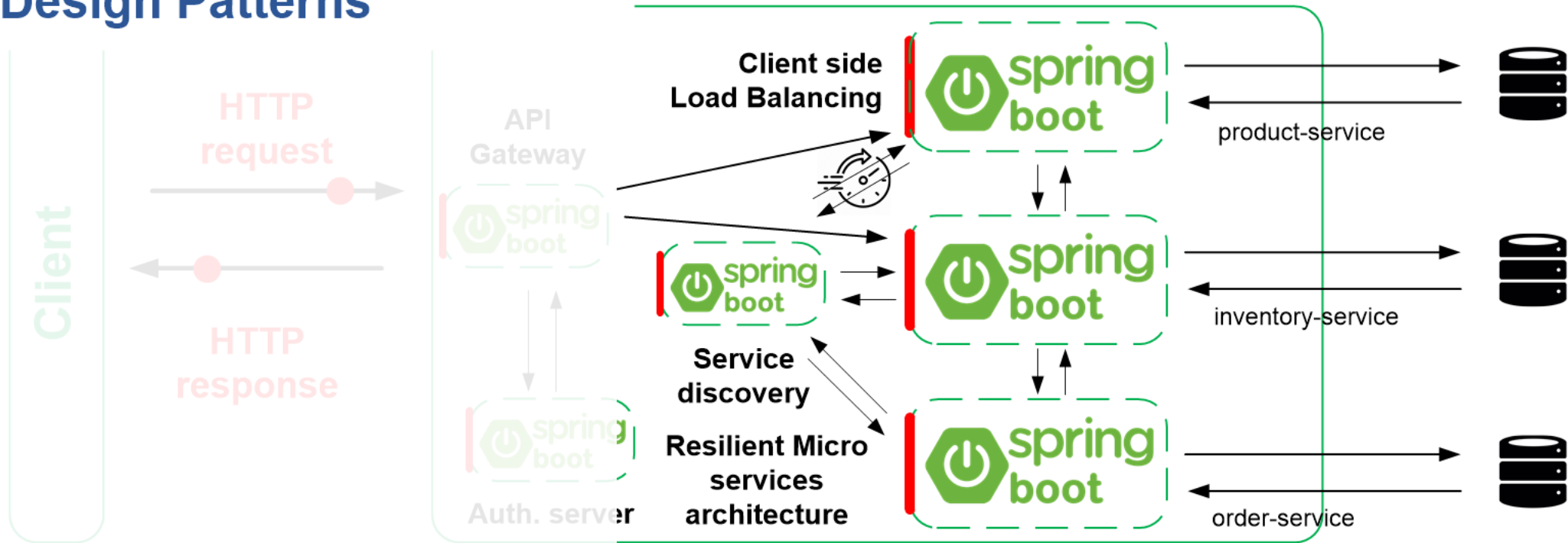
What we have learned?



Implementation technologies we have already learned ...
What we still need to learn ... **Security**

What we have also learned?

Design Patterns



Design Patterns

What we have also learned?

Design Patterns

Service discovery

- Client/Server-side discovery

External API

- API gateway

Reliability/dependability

- Circuit Breaker/fallback mechanisms



Design Patterns

What we have also learned?

Design Patterns

Service discovery

- Client/Server-side discovery

External API

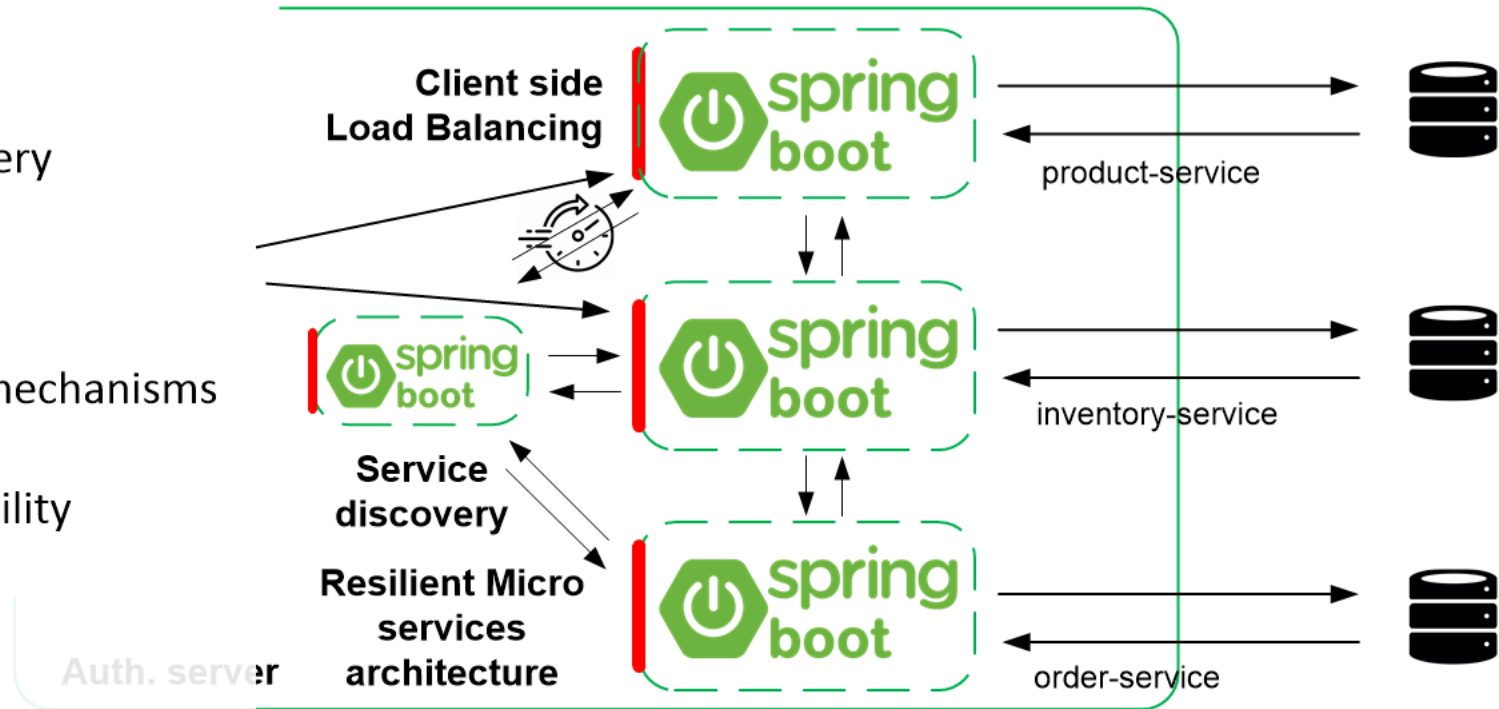
- API gateway

Reliability/dependability

- Circuit Breaker/fallback mechanisms

MS design

- Design by business capability
- Design by subdomain



Design Patterns

What we have also learned?

Design Patterns

Service discovery

- Client/Server-side discovery

External API

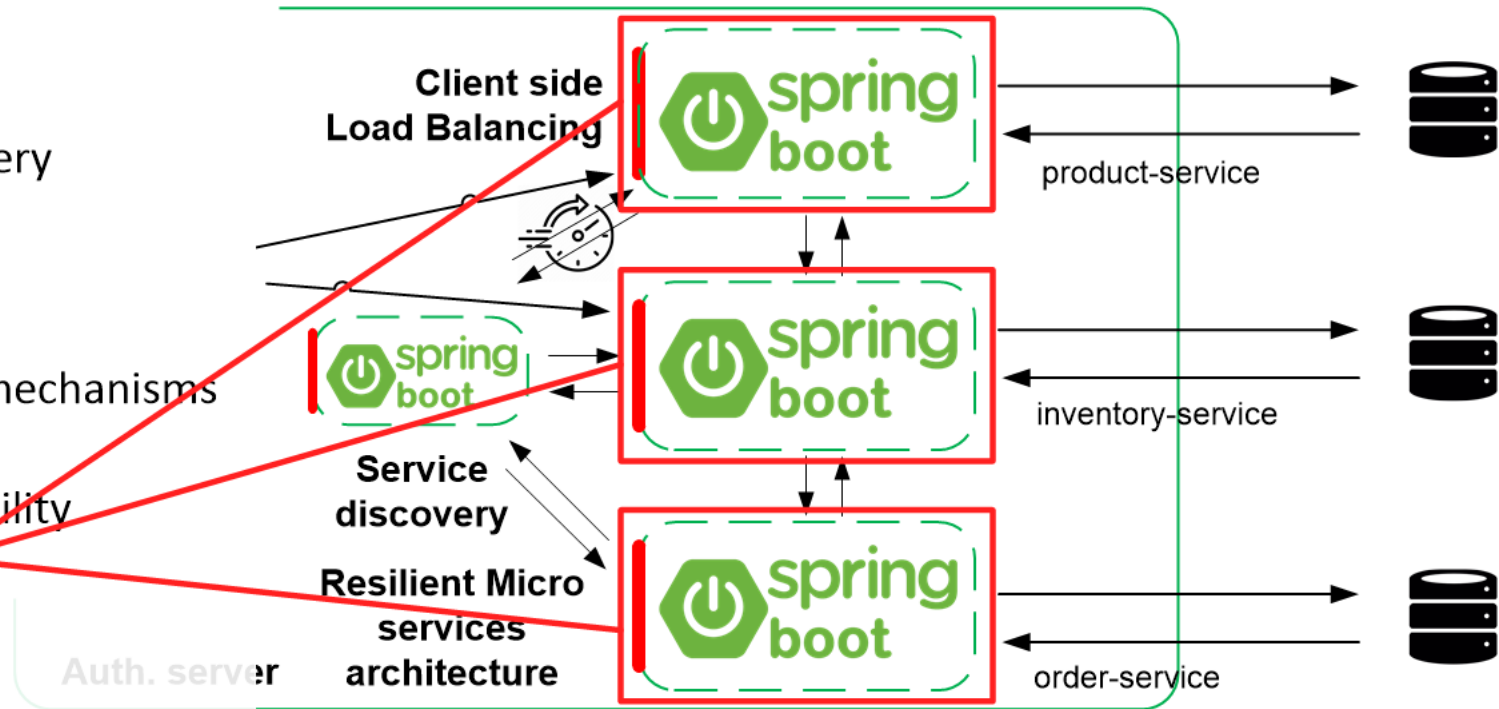
- API gateway

Reliability/dependability

- Circuit Breaker/fallback mechanisms

MS design

- Design by business capability
- Design by subdomain



Design Patterns

What we have also learned?

Design Patterns

Service discovery

- Client/Server-side discovery

External API

- API gateway

Reliability/dependability

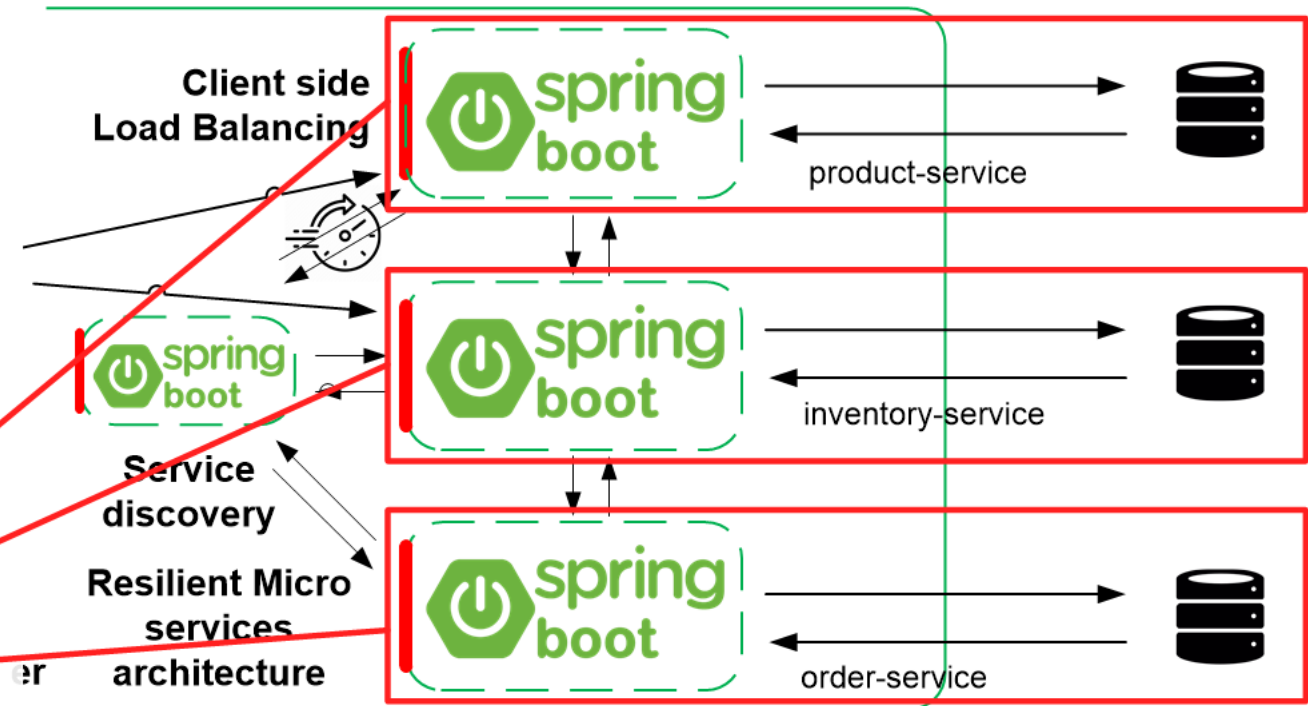
- Circuit Breaker/fallback mechanisms

MS design

- Design by business capability
- Design by subdomain

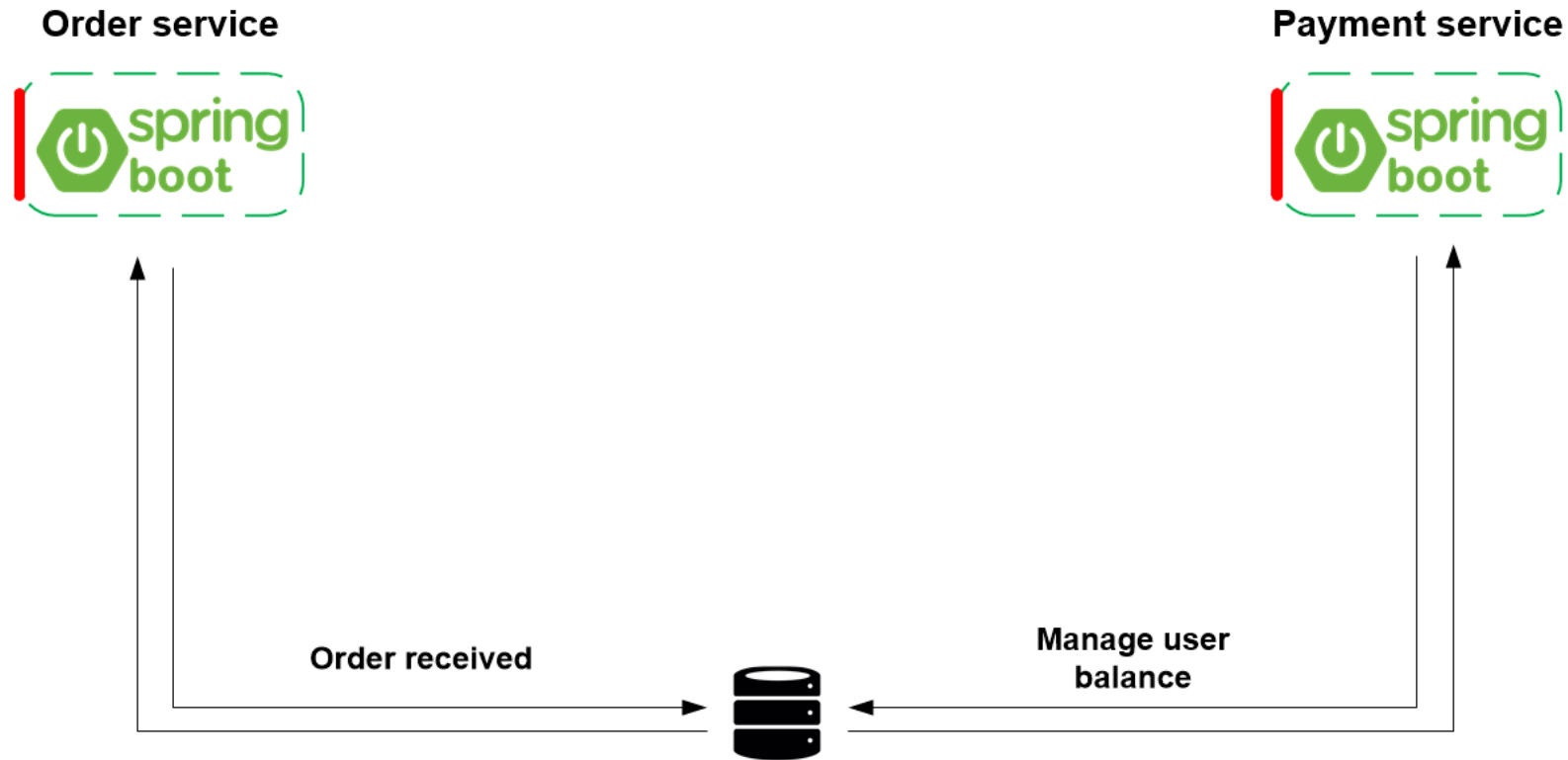
MS Data management

- Database per Service



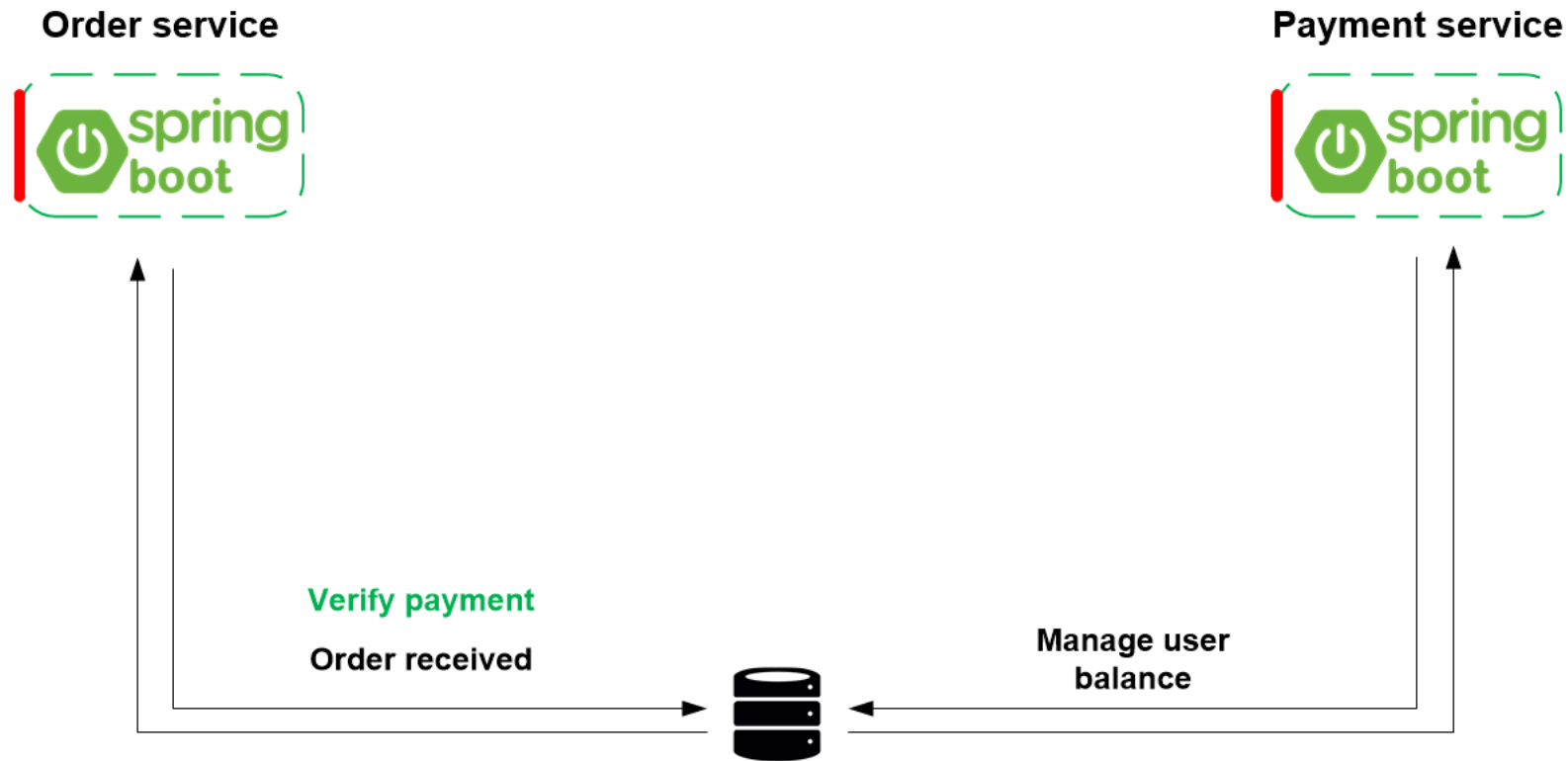
Design Patterns

Database per Service Pattern



Using the Database per Service Pattern will create an issue

Database per Service Pattern



Using the Database per Service Pattern will creates an issue

Database per Service Pattern



Using the Database per Service Pattern will creates an issue

Database per Service Pattern



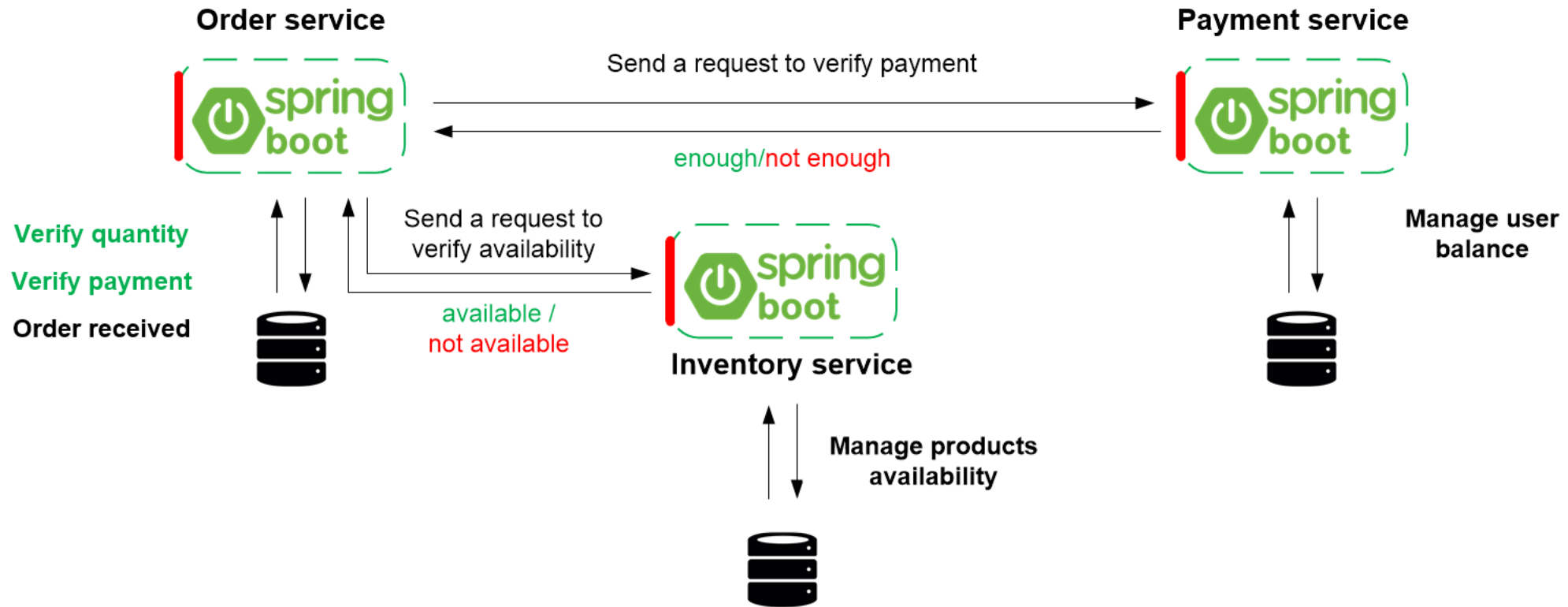
Using the Database per Service Pattern will creates an issue

Database per Service Pattern



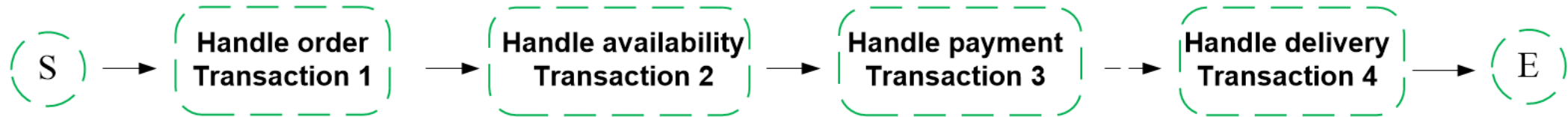
Using the Database per Service Pattern will create an issue
How to implement **transactions** that span across services?

Database per Service Pattern



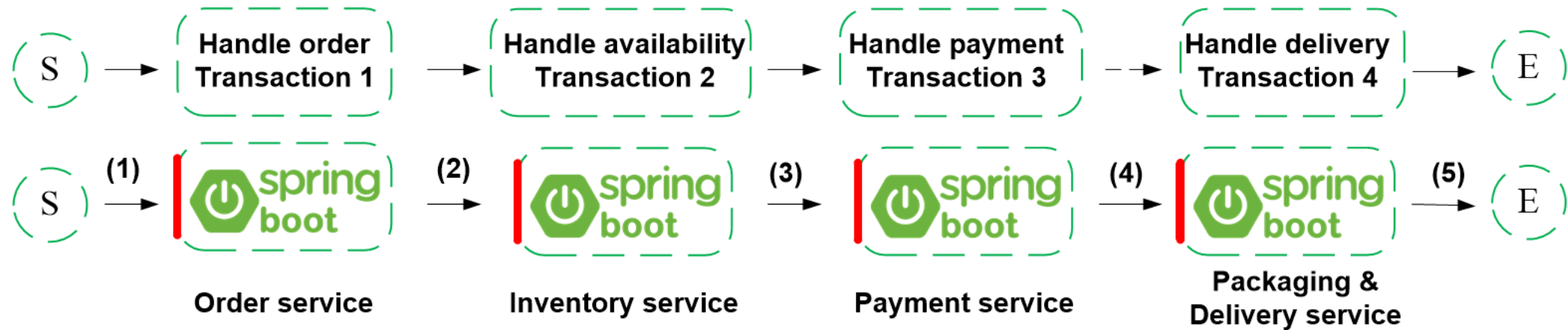
Using the Database per Service Pattern will create an issue
How to implement **transactions** that span across services?

Saga Pattern



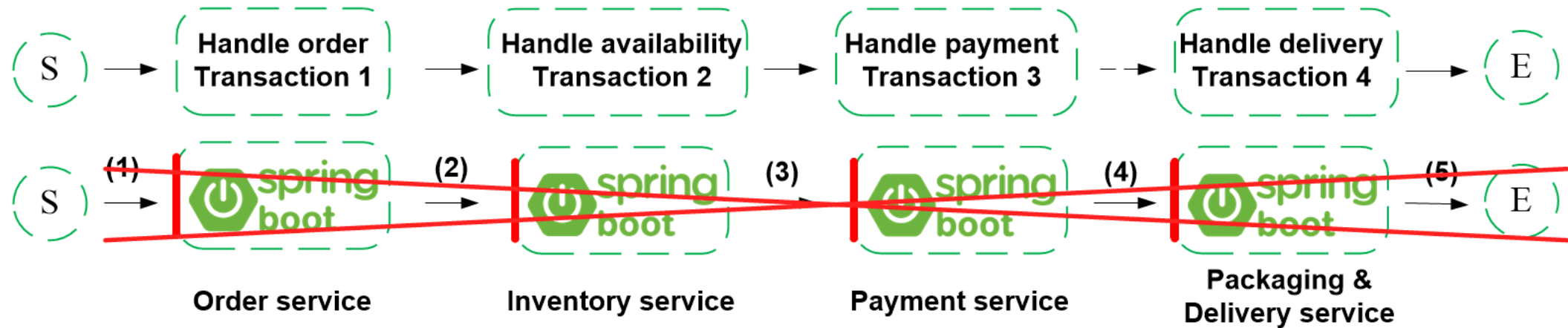
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga Pattern



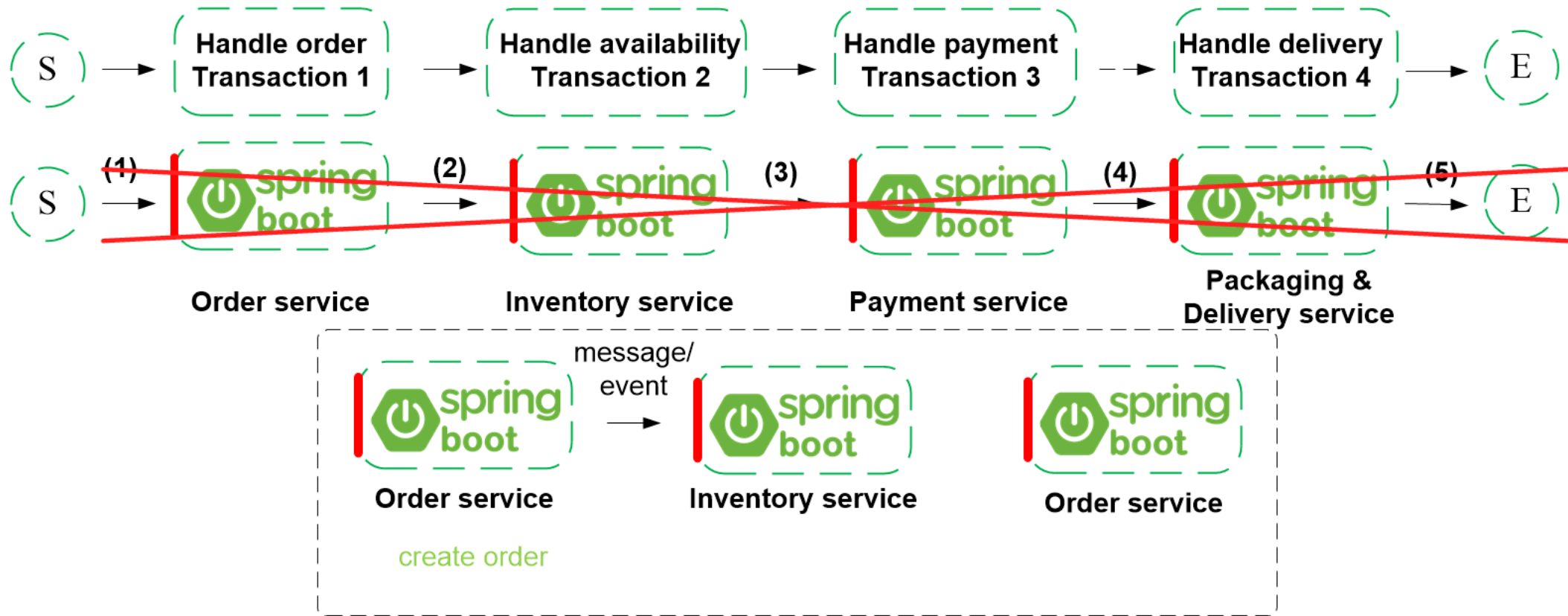
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga Pattern



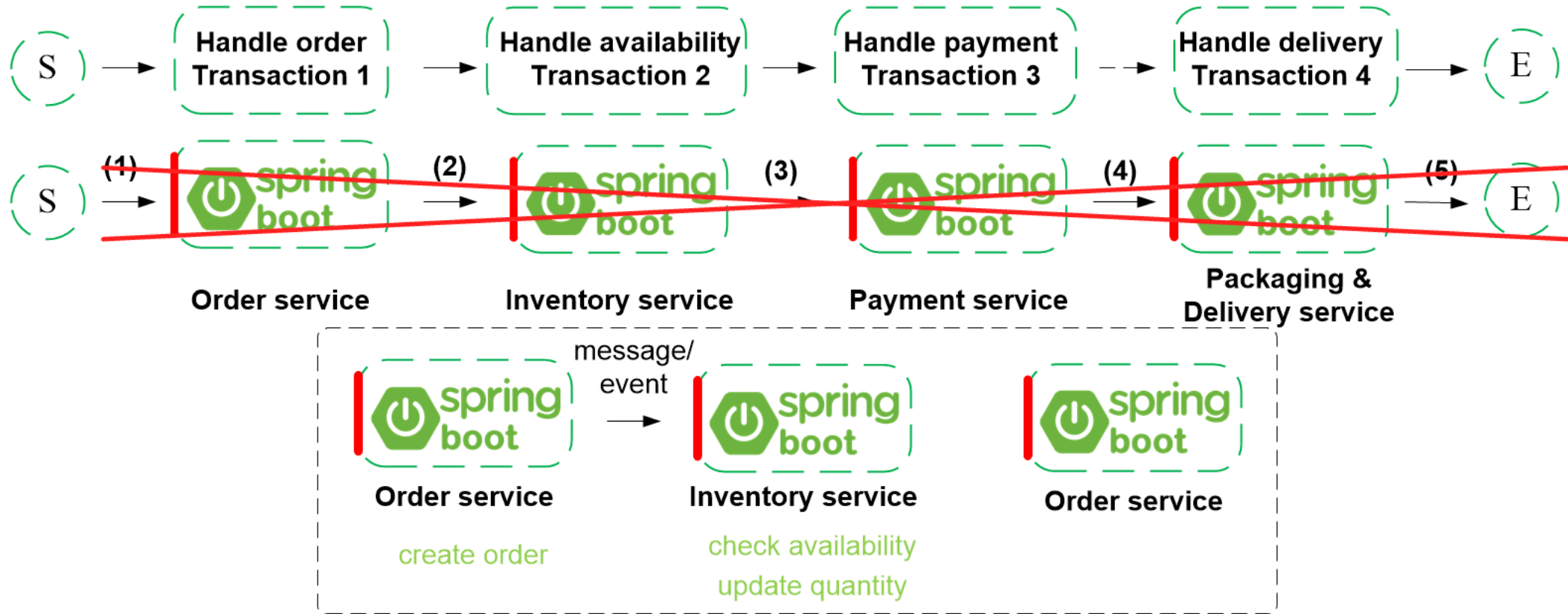
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga Pattern



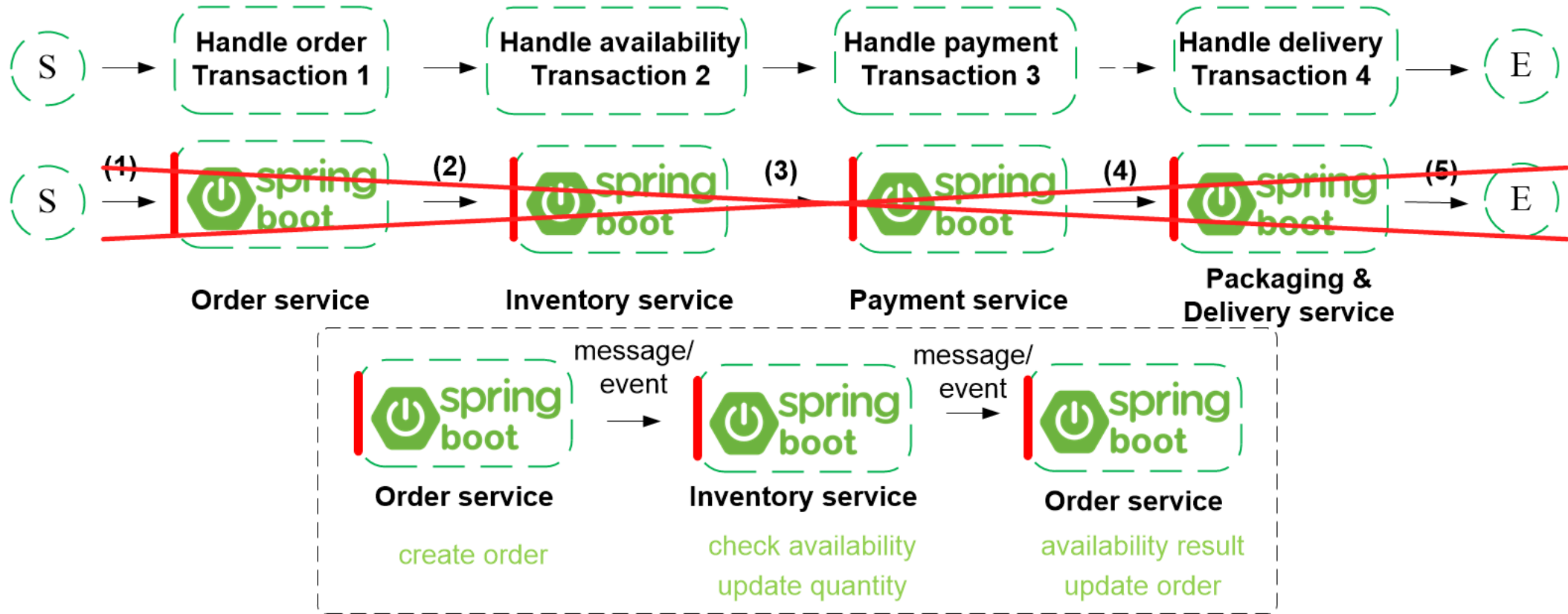
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga Pattern



Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga Pattern



Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Business logic (workflow)



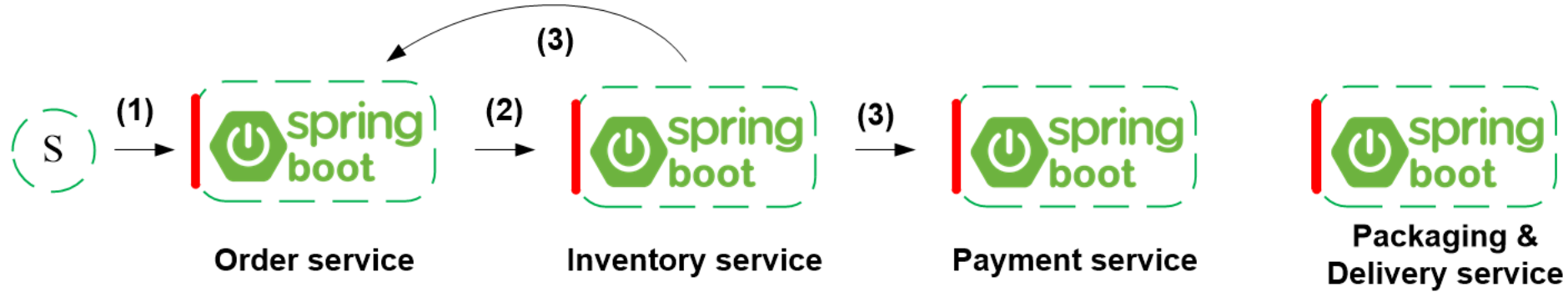
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Business logic (workflow)



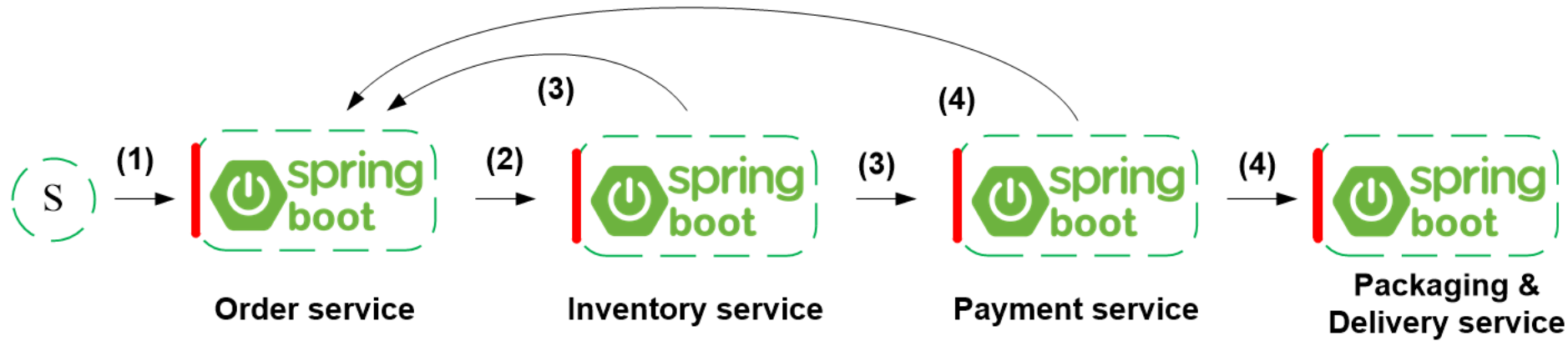
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Business logic (workflow)



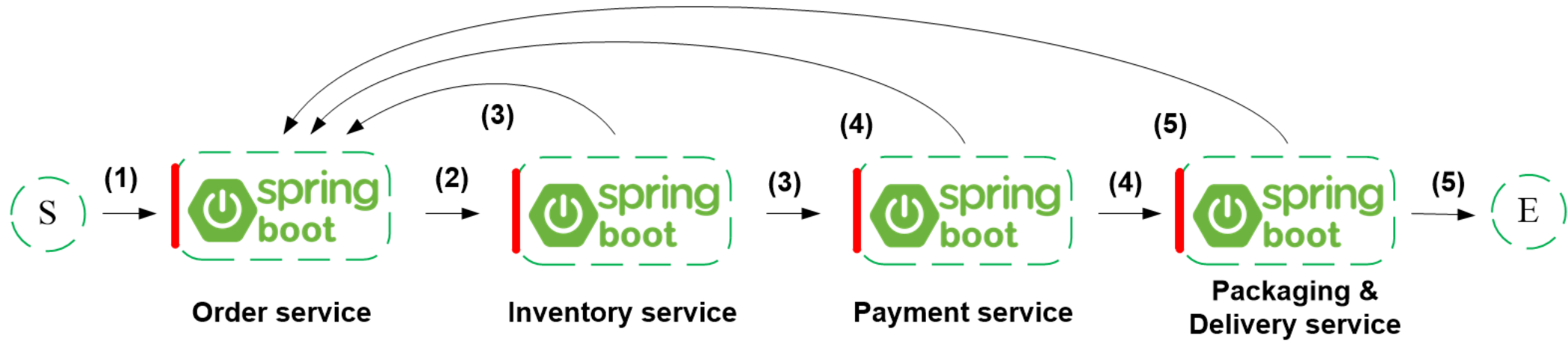
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Business logic (workflow)



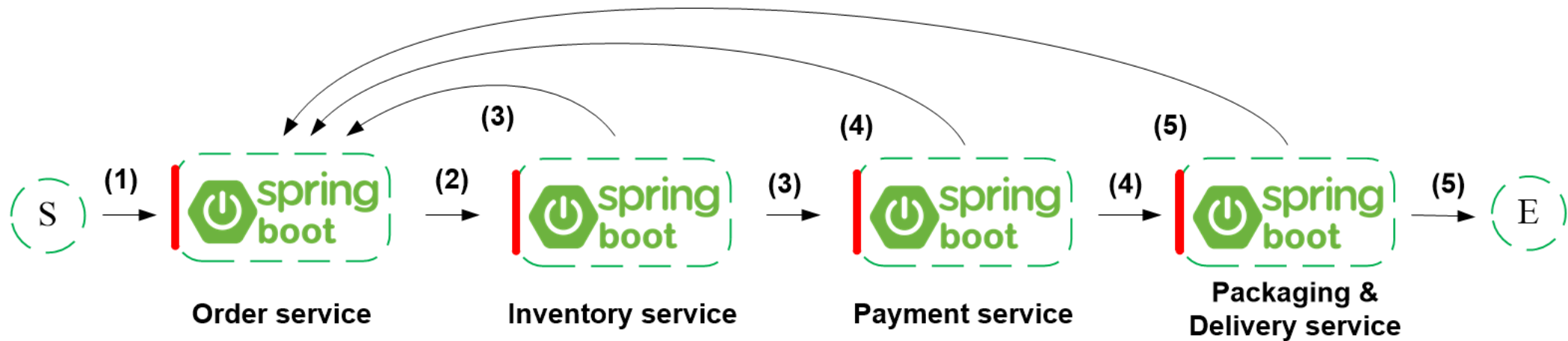
Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Business logic (workflow)



Implement each **business transaction** that spans multiple services as a **saga**. A **saga** is a sequence of local transactions.

Saga - Orchestration vs Choreography



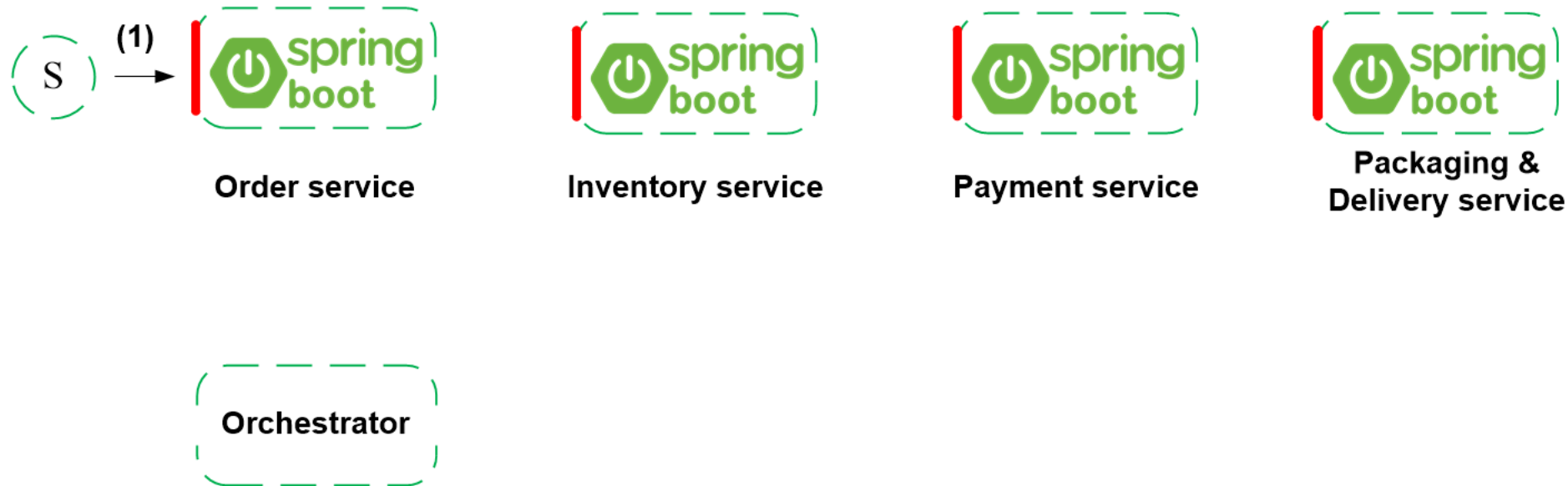
Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Orchestration-based saga

Command-based

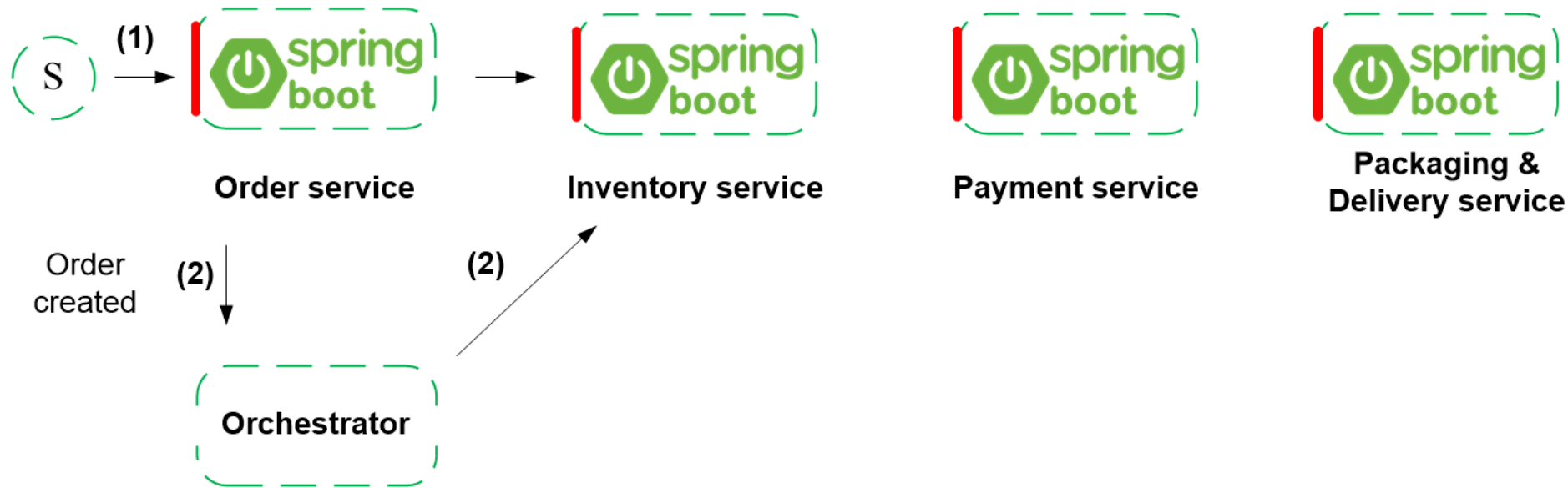
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

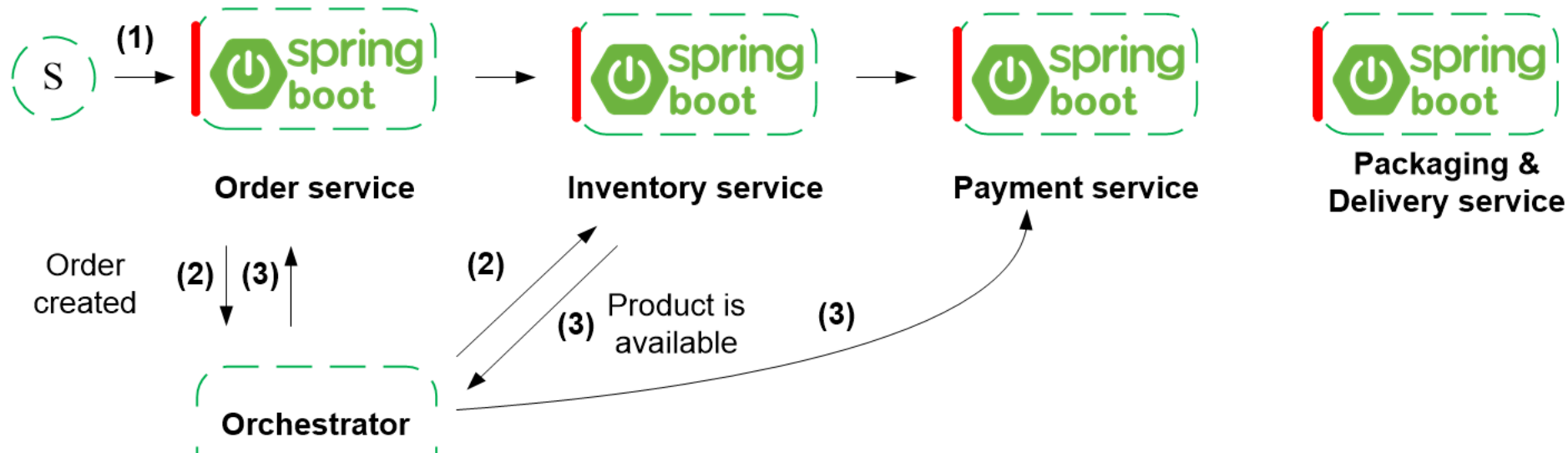
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

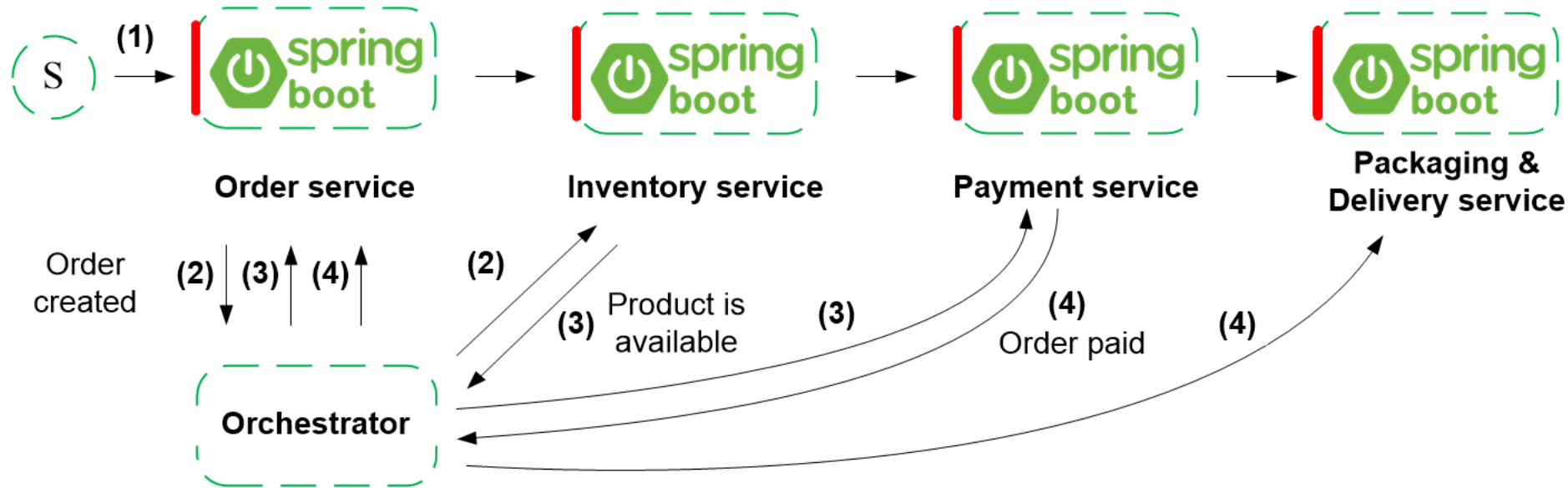
Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



- The Order Service receives the POST /orders request and **creates an Order** in a **PENDING state**. It then **sends a request** (called **command**) to the Inventory Service to **verify** whether the required quantity exist.
- The Inventory Service then sends back a reply message indicating the outcome.
- The saga orchestrator either approves or rejects the Order based on that outcome.

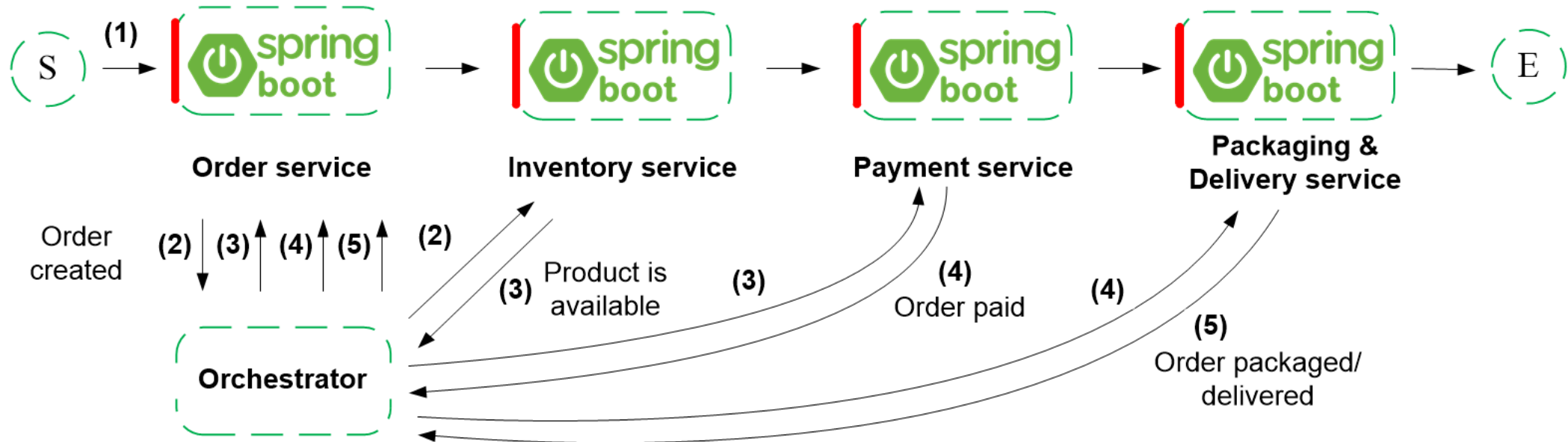
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

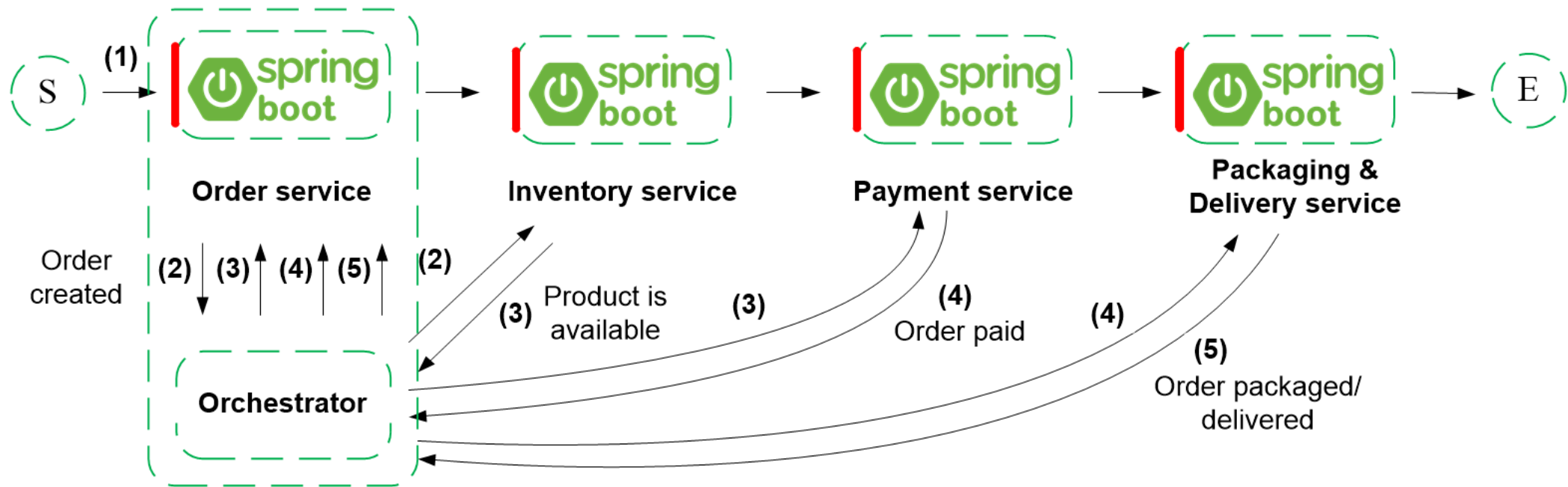
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

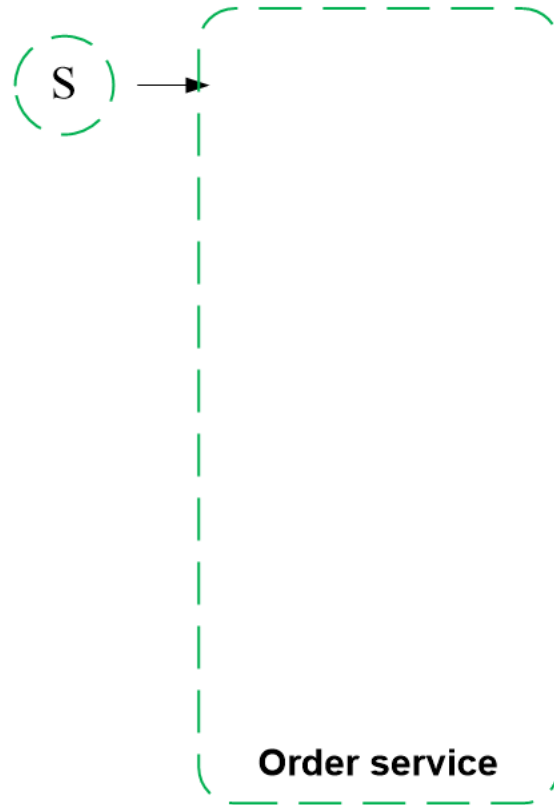
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



Inventory service



Payment service

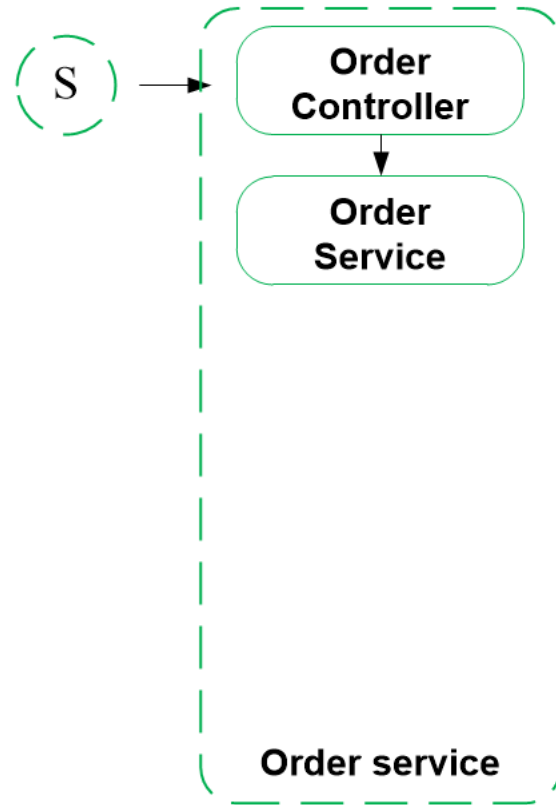


Packaging &
Delivery service

Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



Inventory service



Payment service

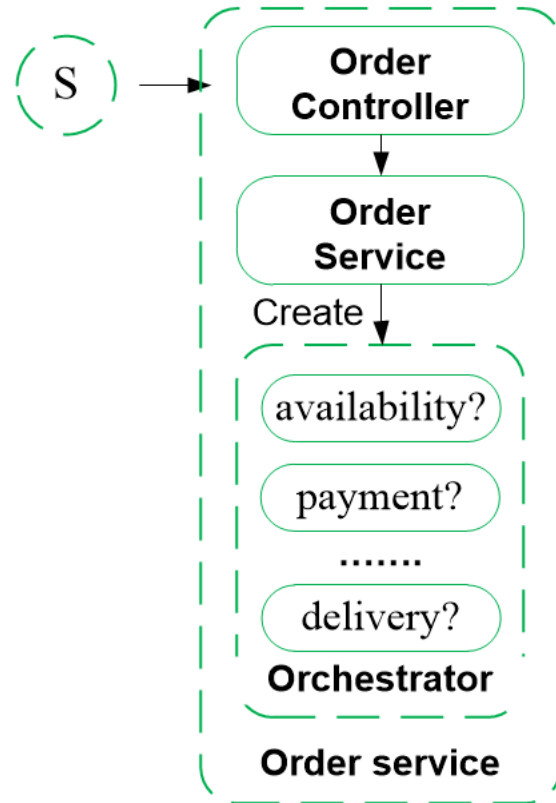


Packaging &
Delivery service

Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

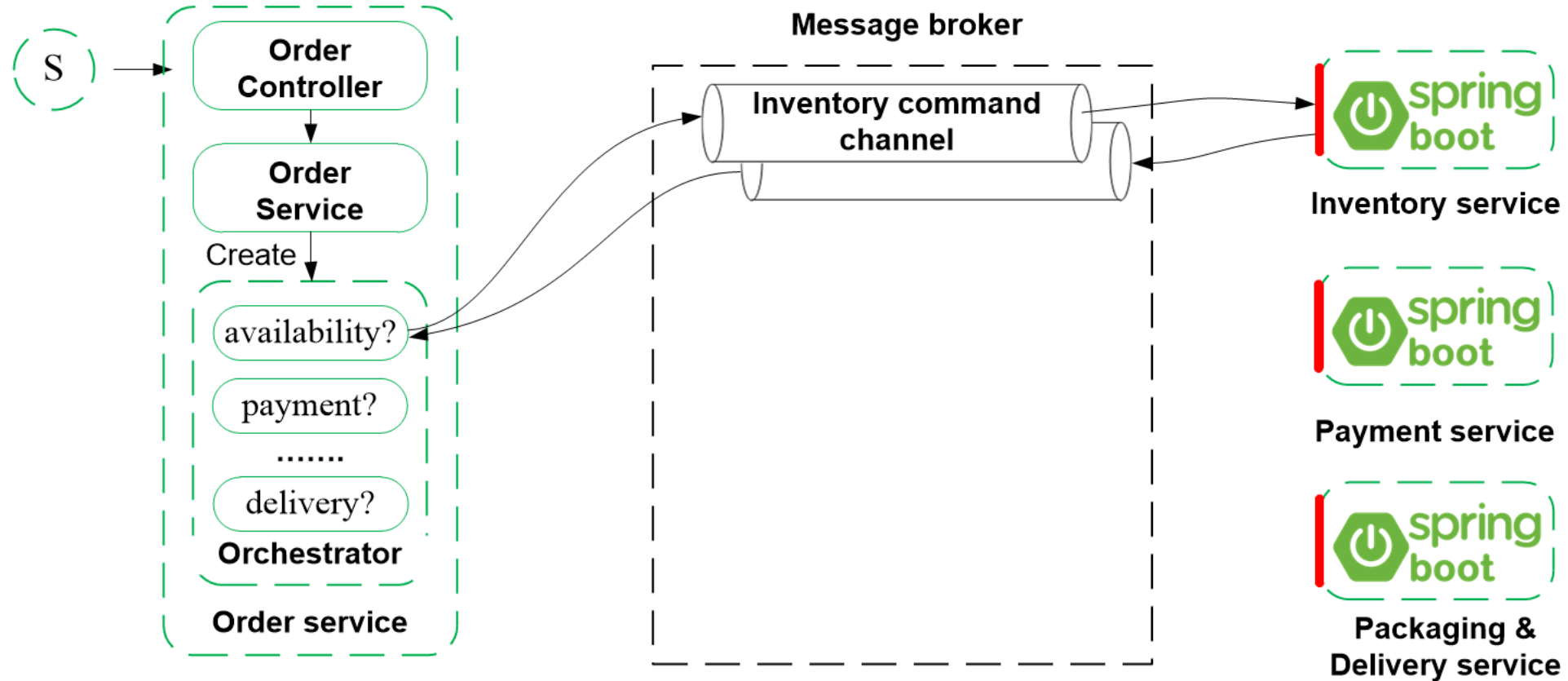
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

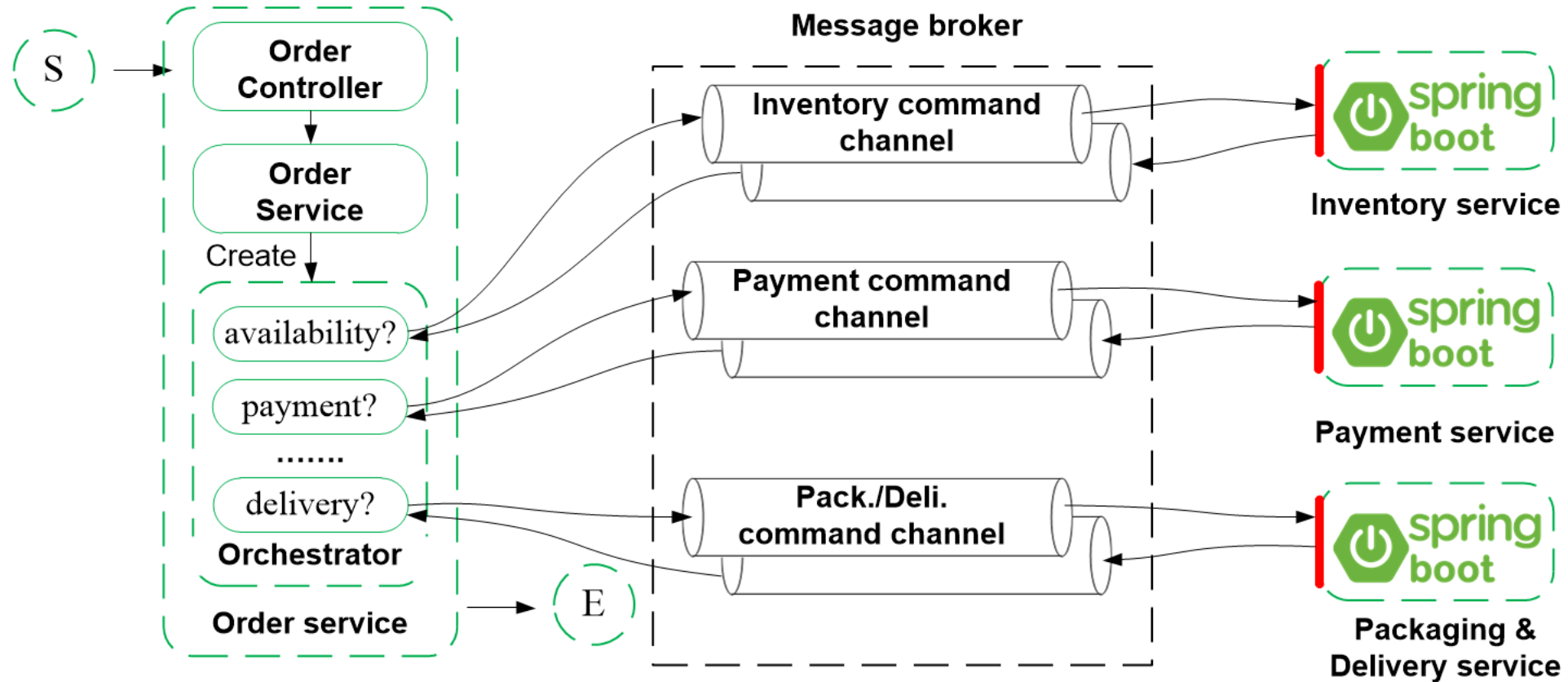
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

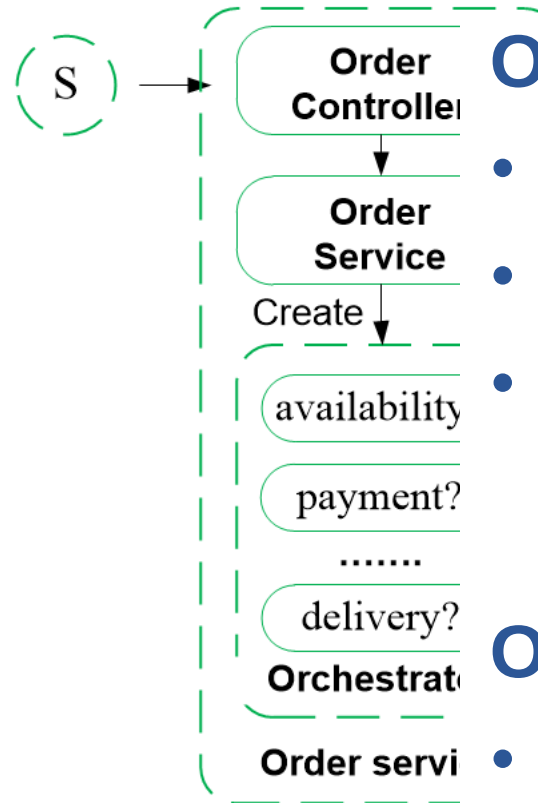
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



Orchestration: Pros

- Services are loosely coupled.
- **No cyclic dependencies.**
- **Simple business logic (separation of concerns):** the "services" do not need to know anything about the sagas that they participate in.

Orchestration : Cons

- Business logic is centralized in the orchestrator.

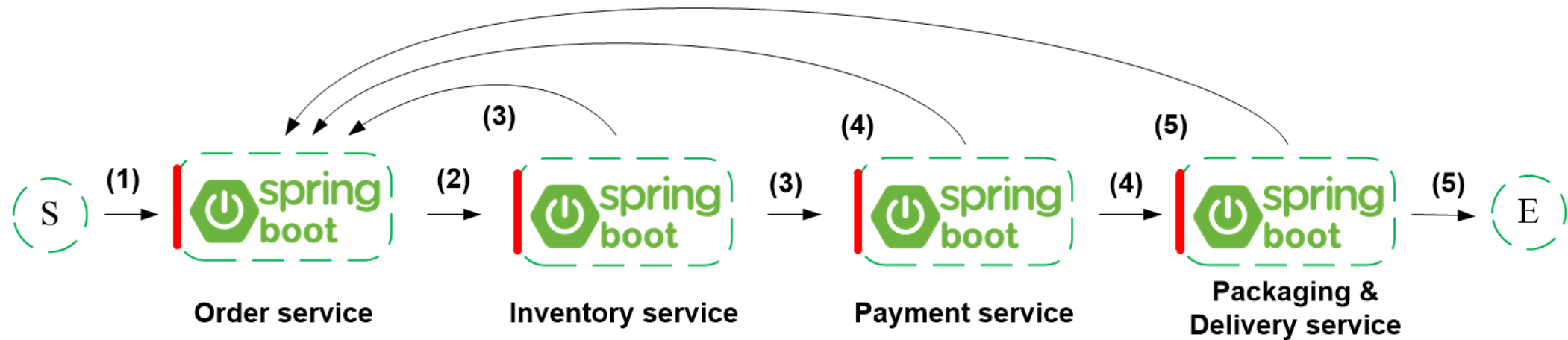
Choreography each
in other services

Orchestration, orchestrator tells participants what transactions to execute

Choreography-based saga

Event-based

Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



Order service



Inventory service



Payment service

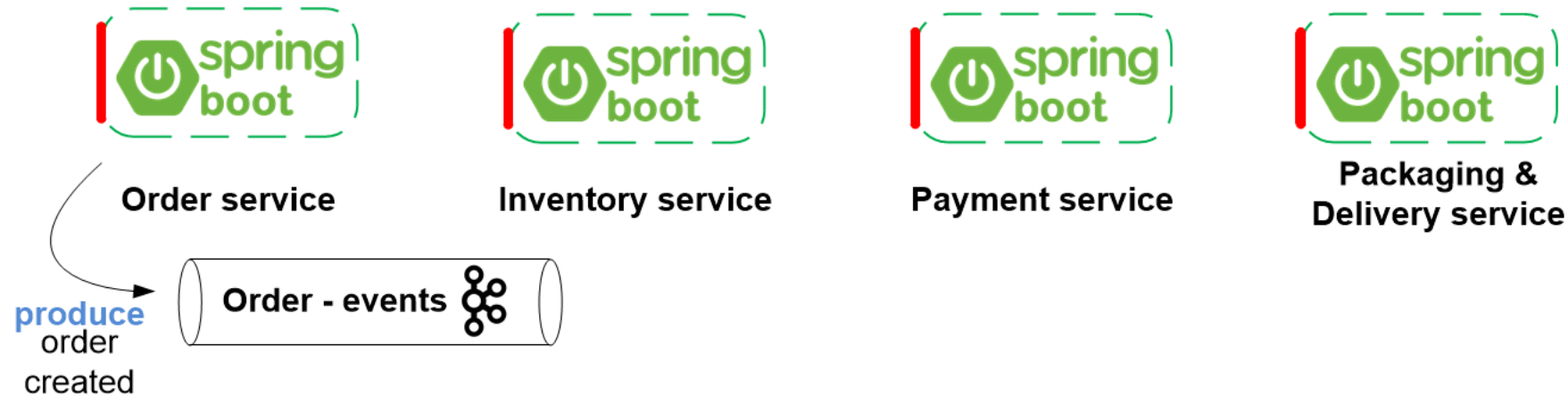


Packaging &
Delivery service

Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

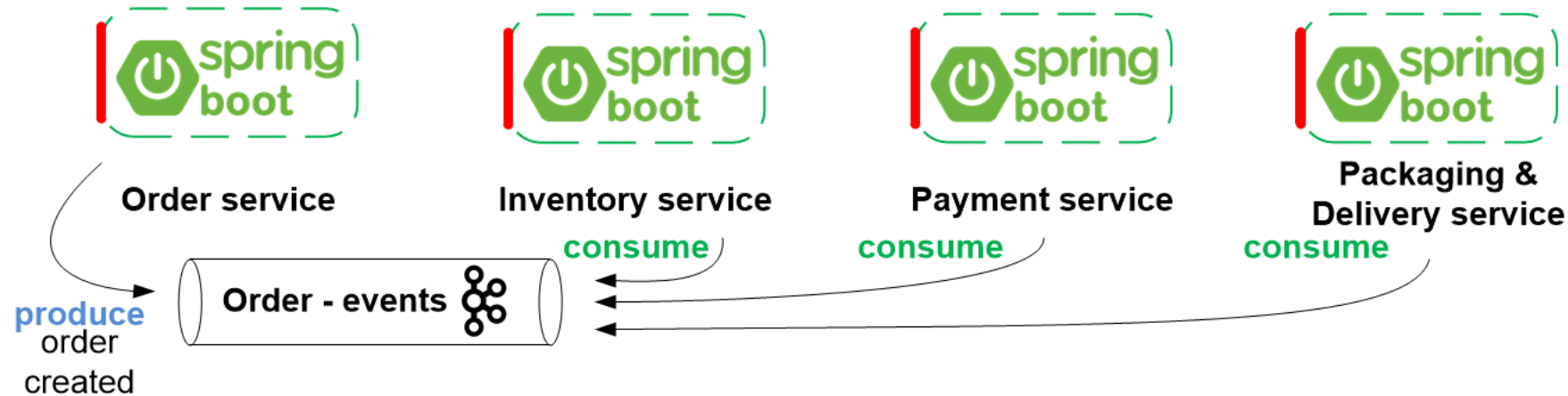
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

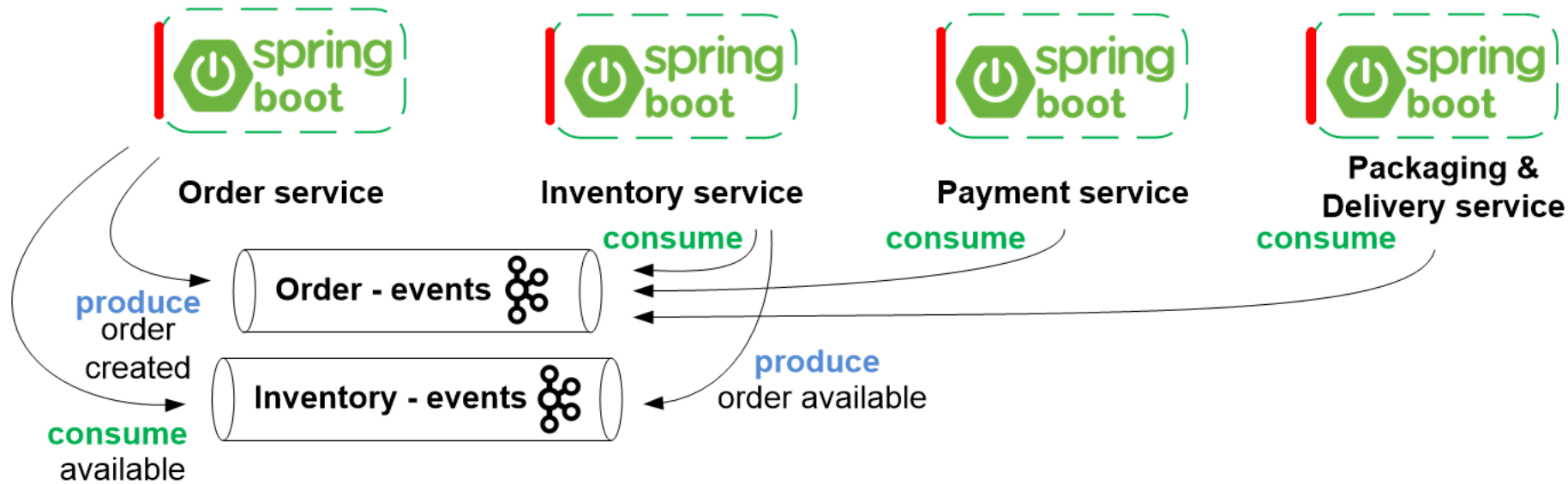
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

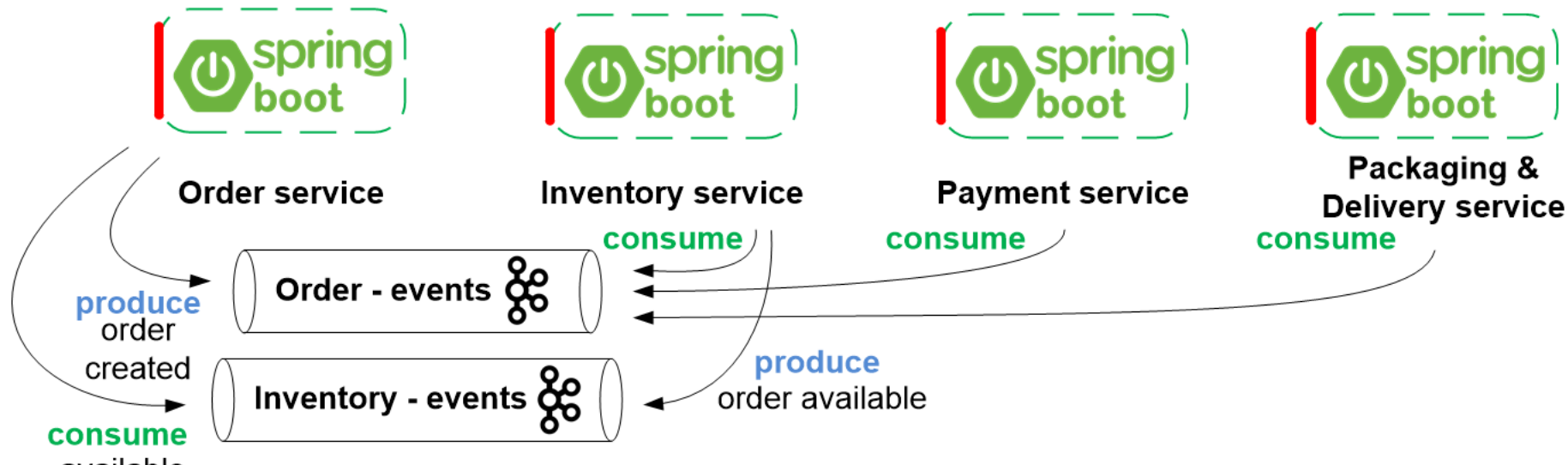
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

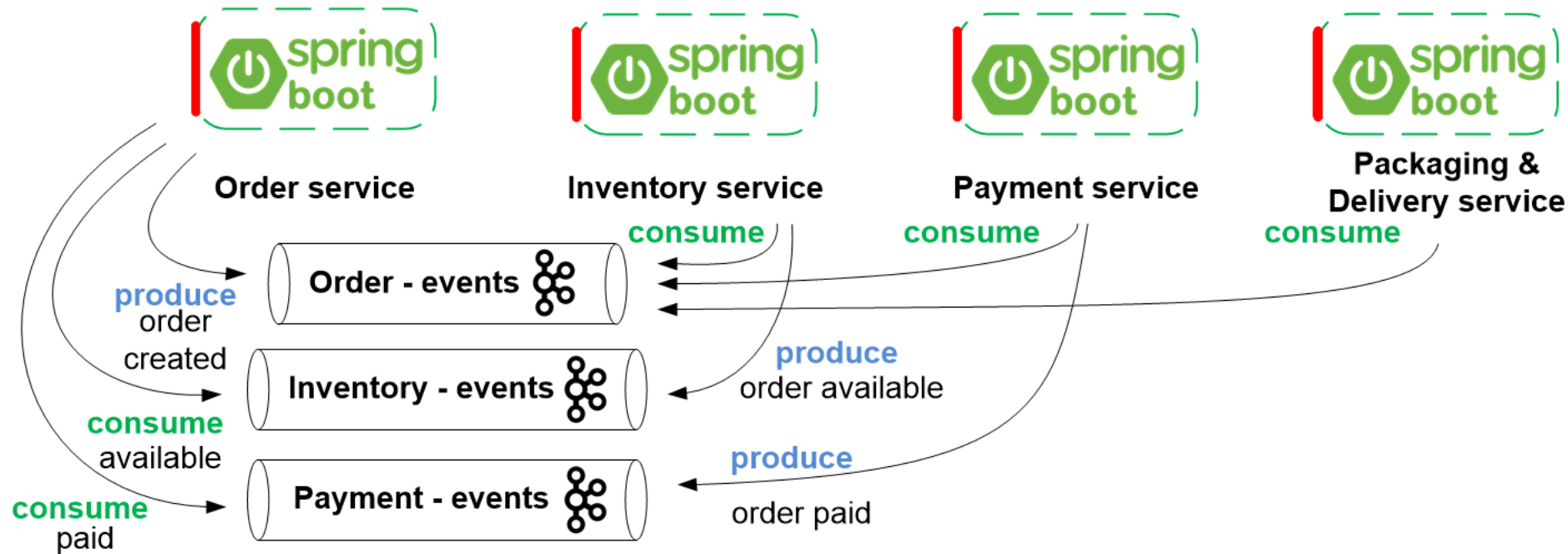
Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



- The Order Service receives the POST /orders request and creates an Order in a **PENDING state**. It then **emits** an **Order Created event**.
- The Inventory Service's event handler **verify** whether the required quantity exist. Then, it **emits** an event indicating the **outcome**
- The Order Service's event handler approves/rejects the order based on that outcome.

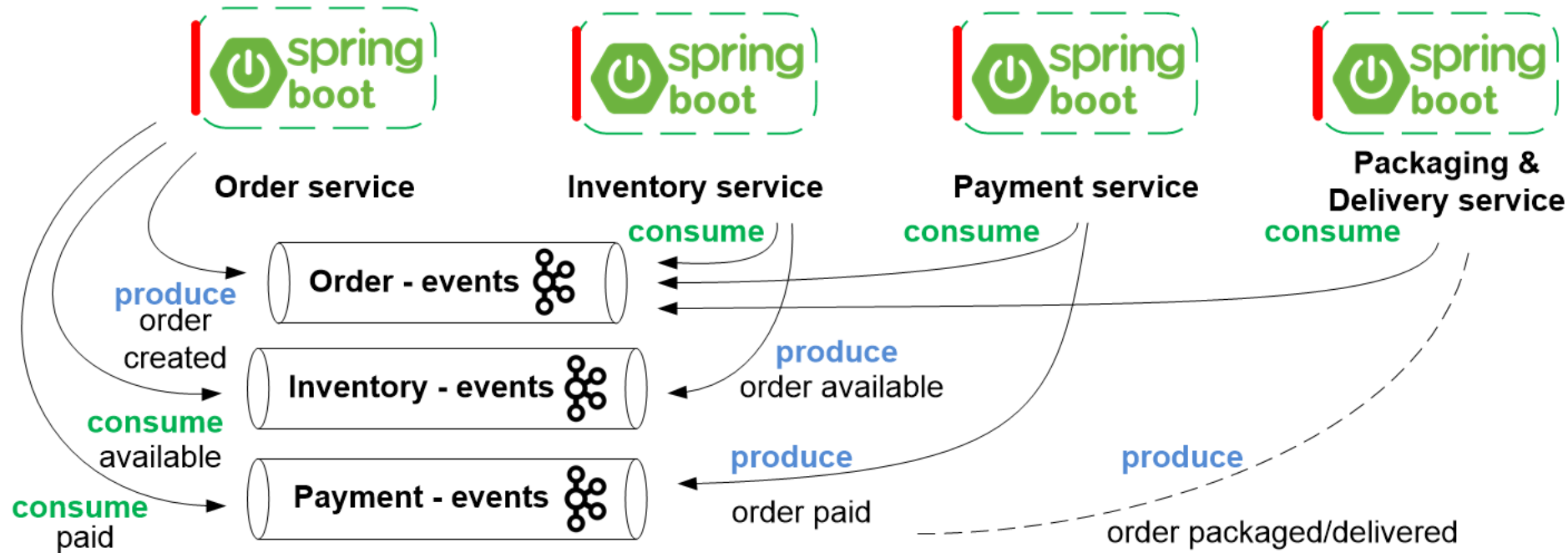
Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Saga - Orchestration vs Choreography



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

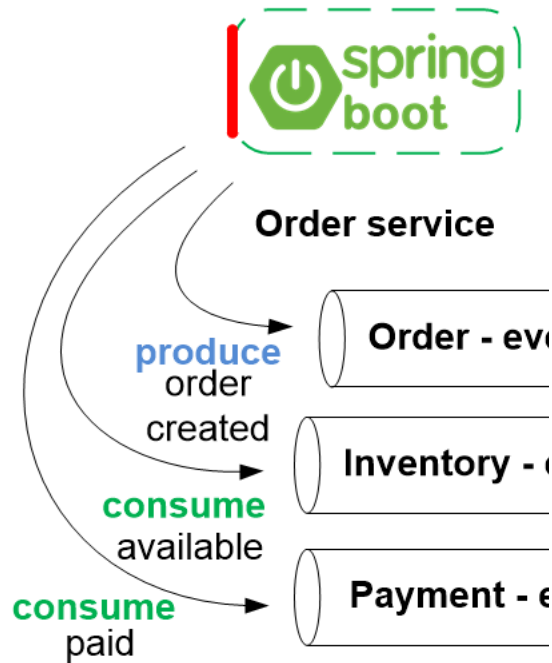
Saga - Orchestration vs Choreography

Choreography: Pros

- Services are loosely coupled;
- Services can be changed independently;
- **Several transactions** can start **simultaneously**.

Choreography: Cons

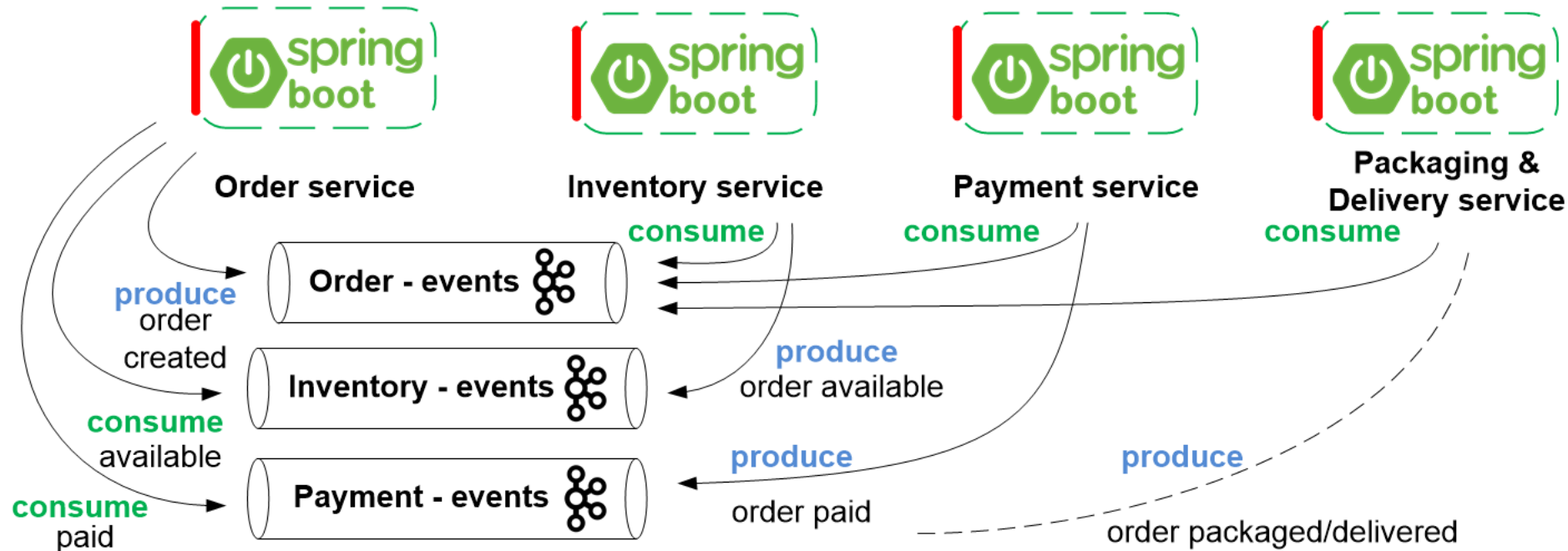
- Difficult to monitor the whole system;
- Retires/timeout can be problematic;
- The business flow is not captured explicitly;
- **Cyclic dependencies between microservices** can be problematic.



Choreography each t
in other services

Orchestration, orchestrator tells participants what transactions to execute

How I can use Choreography-based saga within my project?



Choreography each transaction publishes events that trigger transactions in other services

Orchestration, orchestrator tells participants what transactions to execute

Apache Kafka



Apache Kafka



Kafka is one of the most popular messaging systems (data/events streaming) for **distributed applications**. It is used by thousands of companies today (e.g., Goldman Sachs, Cisco, and Intuit).

Kafka is an integral part of Netflix's real-time monitoring and event-processing pipeline, which processes over **500 billion events a day**. These events include error logs, data on user viewing activities, and troubleshooting events, etc.

Kafka is one of the most popular messaging systems (data/events streaming) for **distributed applications**. It is used by thousands of companies today (e.g., Goldman Sachs, Cisco, and Intuit).

Kafka is an integral part of Netflix's real-time monitoring and event-processing pipeline, which processes over **500 billion events a day**. These events include error logs, data on user viewing activities, and troubleshooting events, etc.

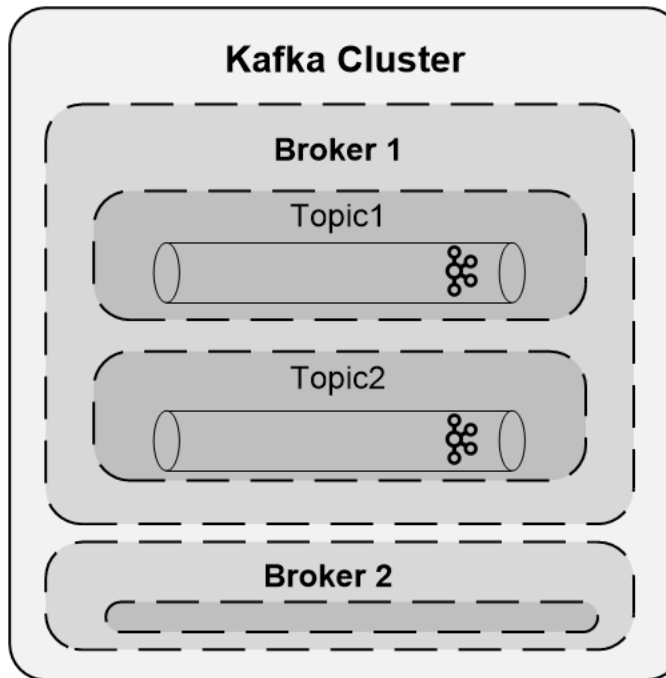
Why is Apache Kafka so popular?

Kafka is **highly scalable** without any downtime impact, it has a **distributed architecture** capable of handling messages with higher volume and velocity.

Kafka offers a very low **latency**, which is as low as ten milliseconds.

Kafka is a very **reliable** system to use as it is a distributed platform with very high **fault tolerance**.

Apache Kafka - Architecture

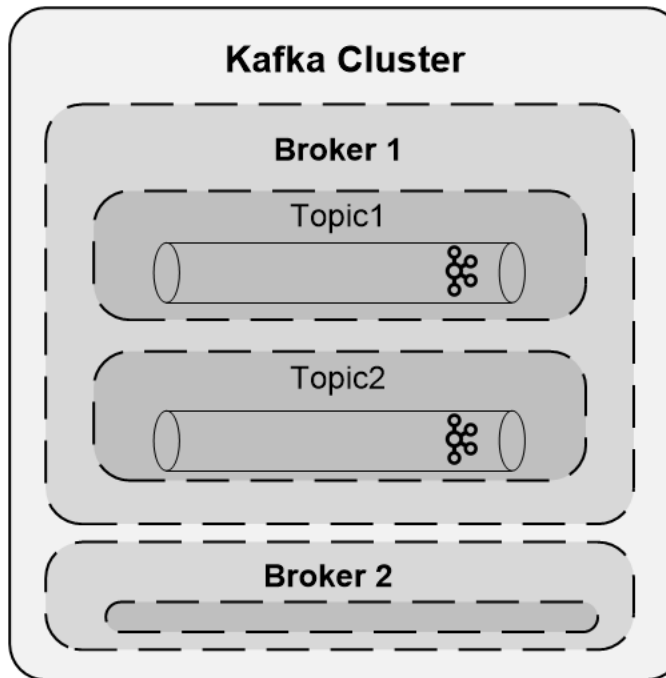


A topics is a stream of **messages** that belong to a specific **category**

A broker works as a container that can hold multiple topics with different partitions.

A cluster comprises one or more **brokers**.

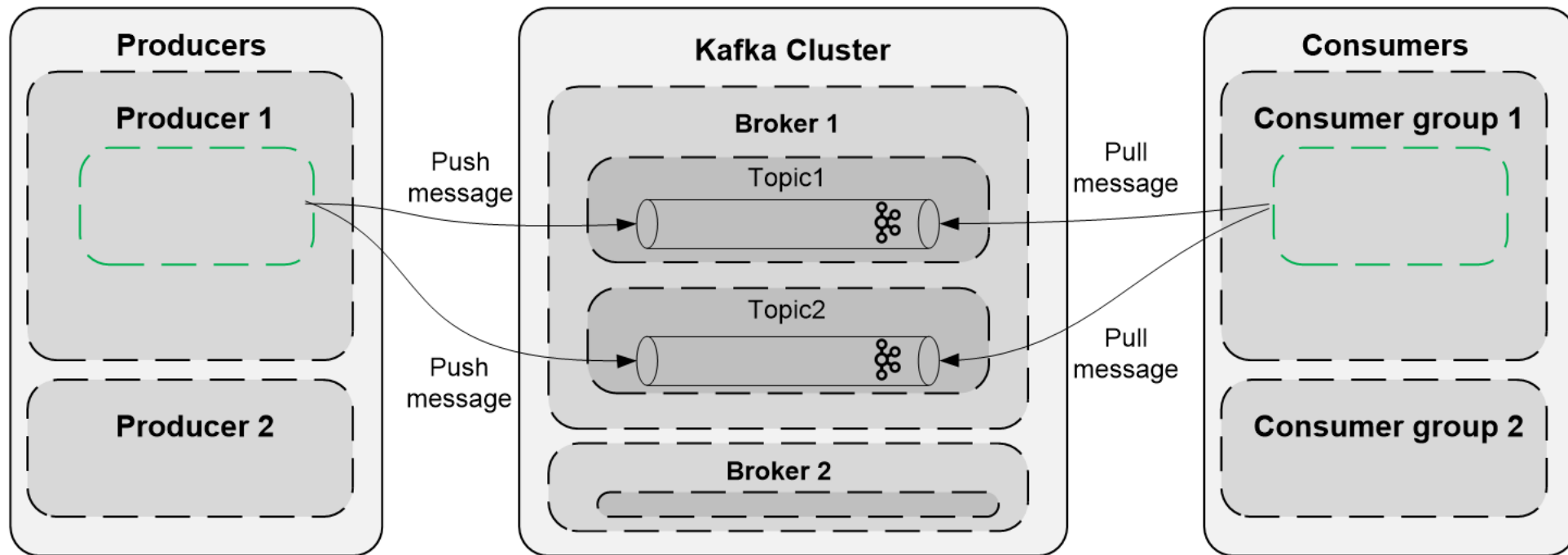
Apache Kafka - Architecture



Topics are divided into a configurable number of parts, which are known as **partitions**.

Partition Offset - Messages or records in Kafka are assigned to a partition. To specify the position of the records within the partition, each record is provided with an offset.⁵⁷

Apache Kafka - Architecture

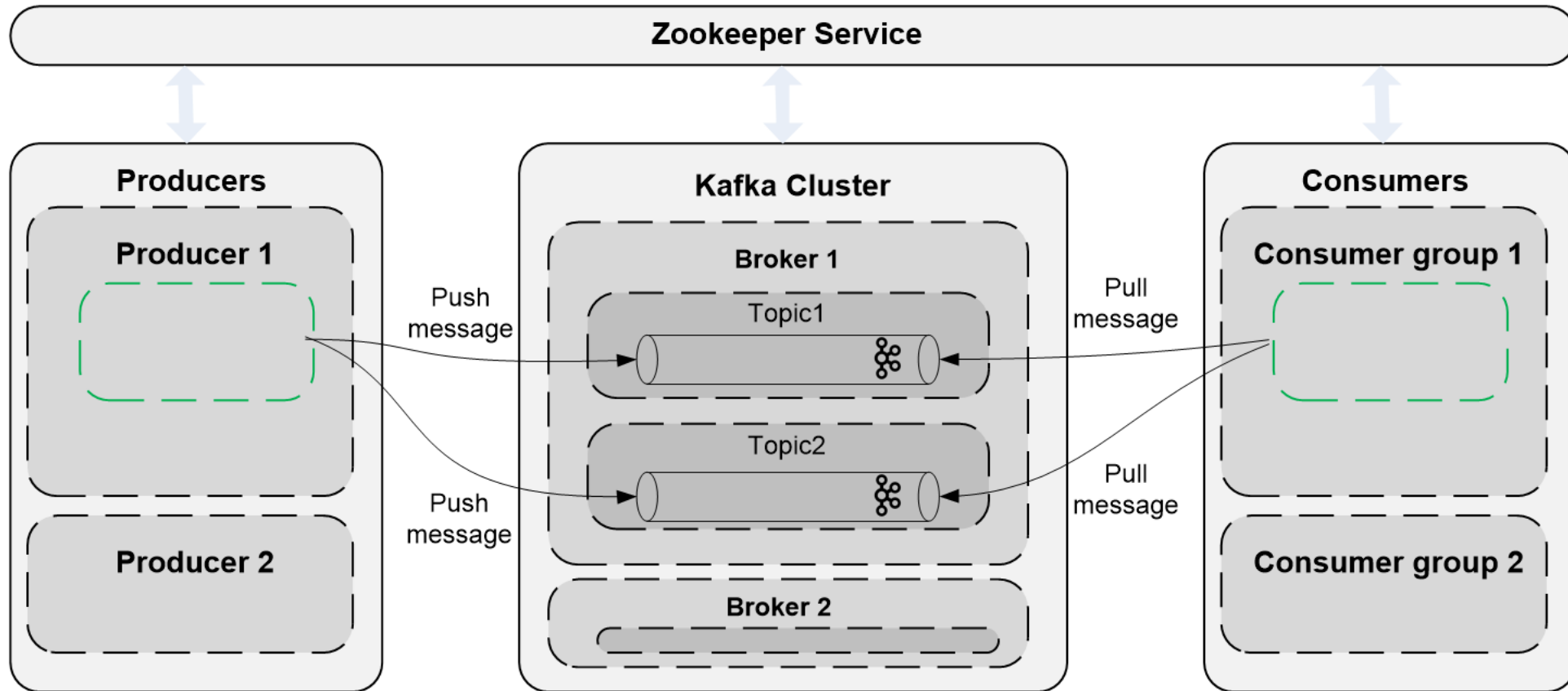


A **producer** publishes (push) messages to one or more **topics**.

A **consumer** read (pull) data from the Kafka **cluster**.

A **consumer group** refers to a number of consumers that pull data from the same **topic**

Apache Kafka - Architecture



Apache ZooKeeper is a software developed by Apache that acts as a centralized service to manage and coordinate Kafka brokers in the cluster.

CQRS Pattern

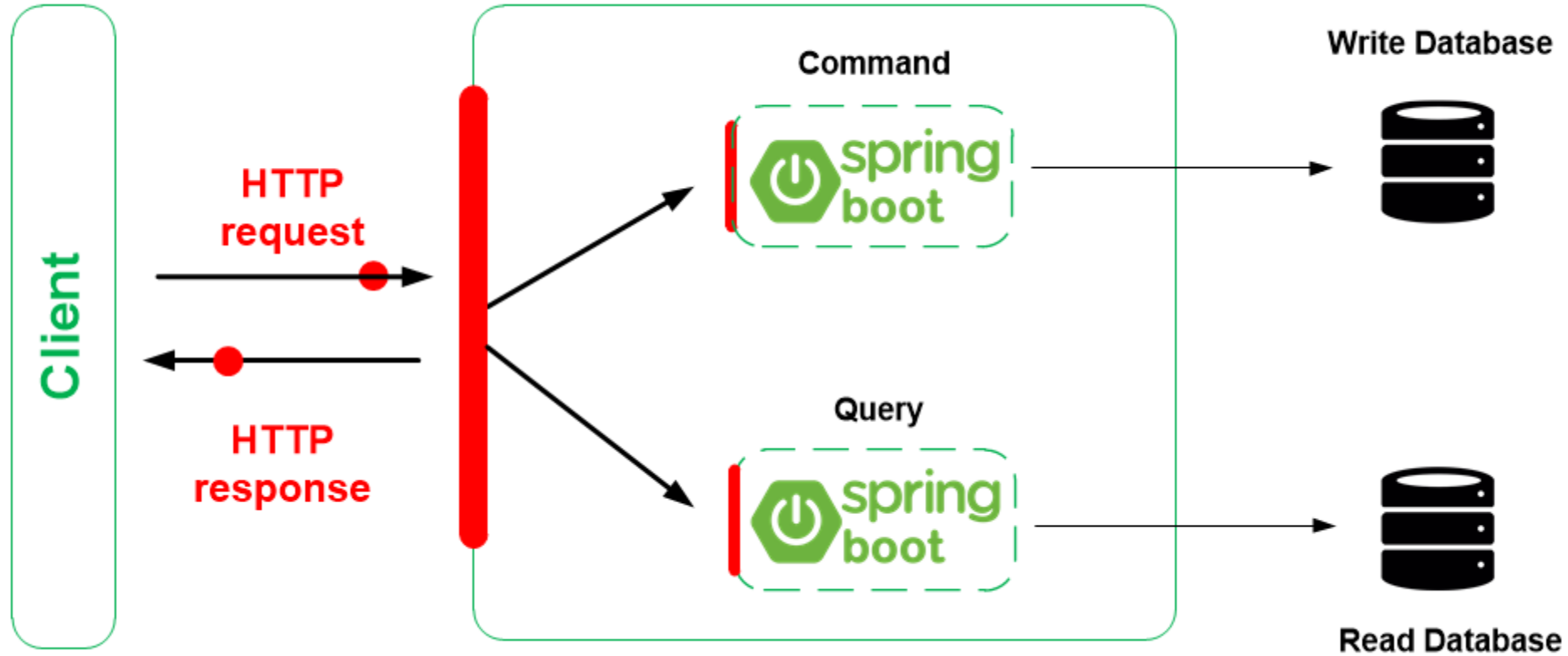
Command and Query Responsibility Segregation

CQRS Pattern



Command and Query Responsibility Segregation (CQRS) is a design pattern that offers to use “**separation of concerns**” principles and separate reading database and the writing database.

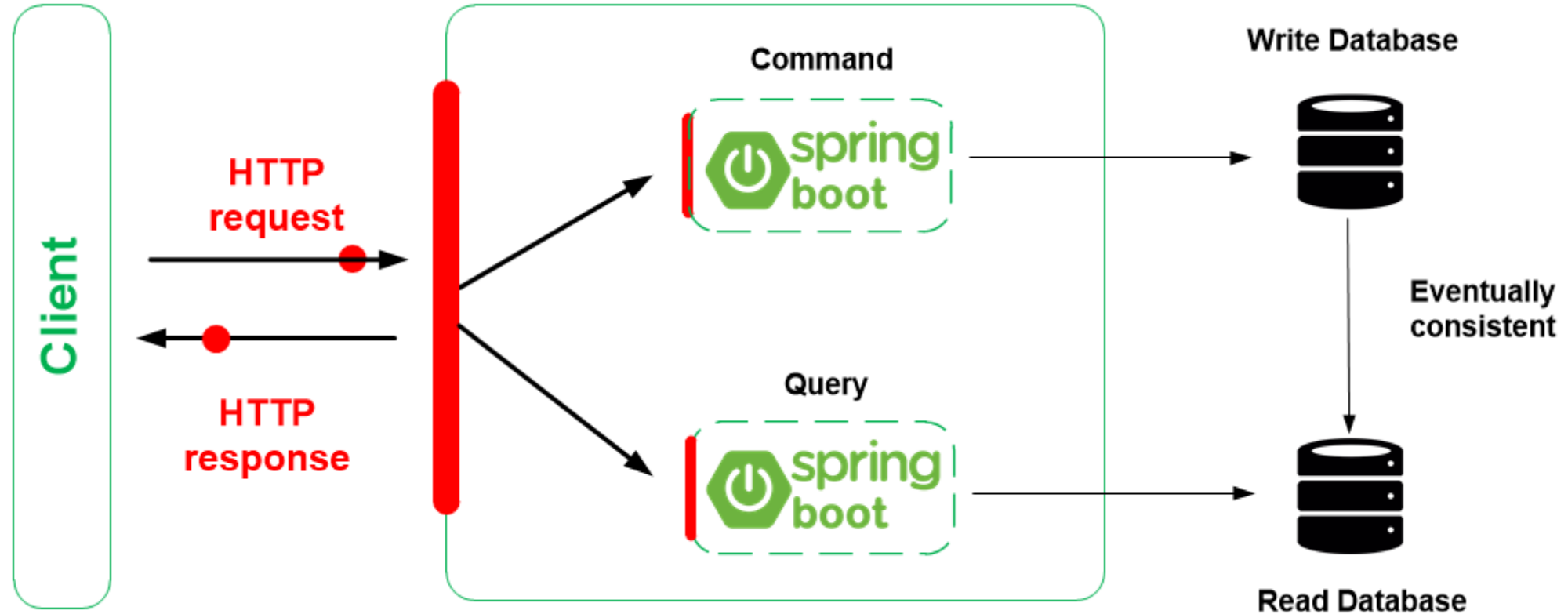
CQRS Pattern



Commands are actions that change the state of a database (e.g., add/modify/delete an order).

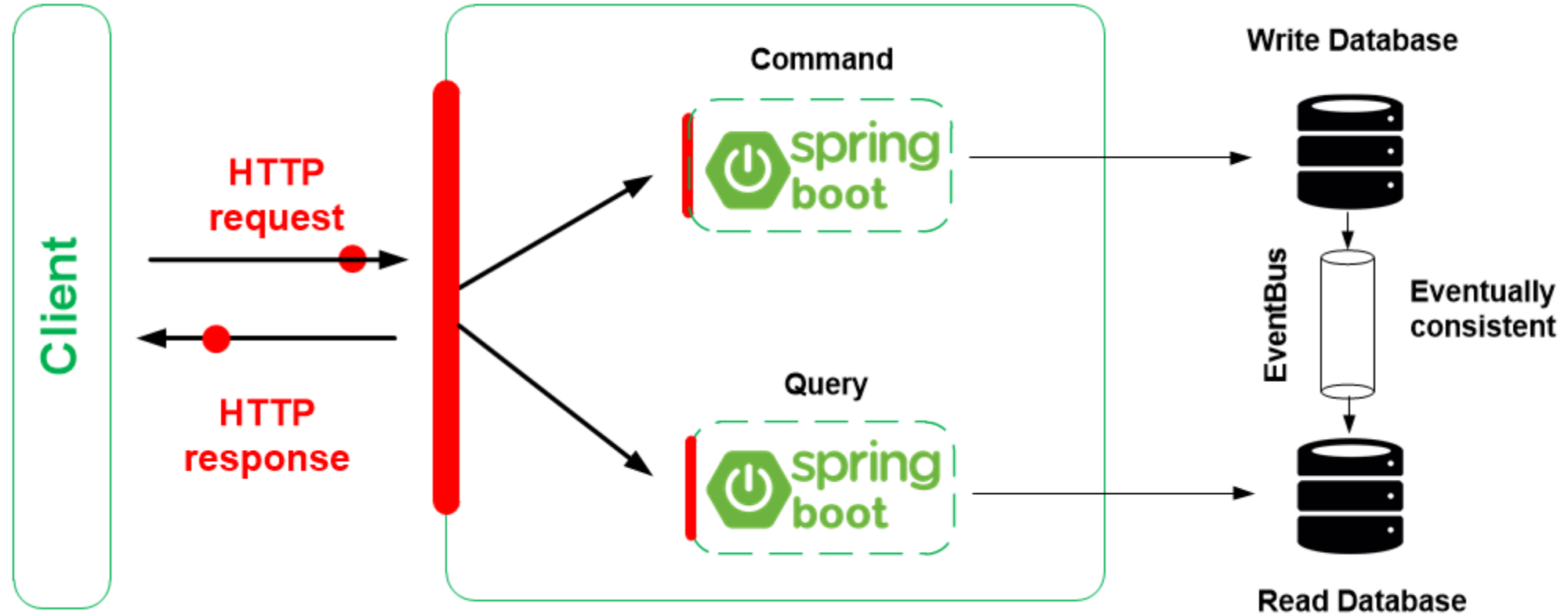
Queries never change the state of a database (e.g., fetch an order).

CQRS Pattern



Command and Query Responsibility Segregation (CQRS) is a design pattern that offers to use “**separation of concerns**” principles and separate reading database and the writing database.

CQRS Pattern



Command and Query Responsibility Segregation (CQRS) is a design pattern that offers to use “**separation of concerns**” principles and separate reading database and the writing database.

Reactive Streams API - recap

Reactive Streams API - recap

The new Reactive Streams API was created by engineers from Netflix, Pivotal, Lightbend, RedHat, Twitter, and Oracle, among others and are now part of Java 9.

It defines four interfaces:

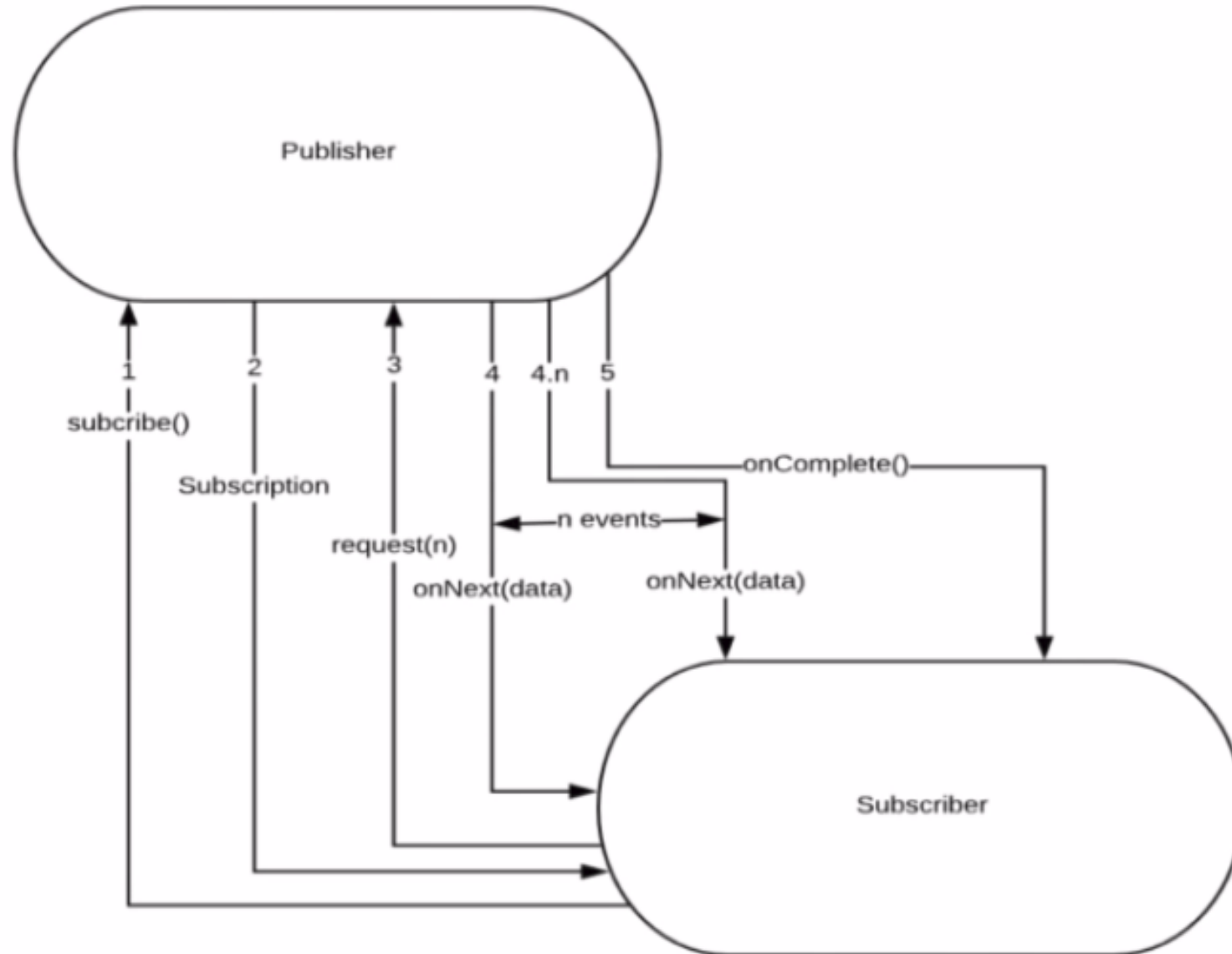
Publisher: **Emits** a sequence of **events** to **subscribers** according to the demand received from its **subscribers**. **A publisher can serve multiple subscribers.**

Subscriber: **Receives** and **processes events** emitted by a **Publisher**.

Subscription: Defines a **one-to-one relationship** between a **Publisher** and a **Subscriber**.

Processor: Represents a processing stage consisting of both a **Subscriber** and a **Publisher** and obeys both contracts.

Reactive Streams API - recap



What is Spring WebFlux? - recap

Spring WebFlux is a parallel version of Spring MVC and supports fully non-blocking reactive streams. It supports the back pressure concept and uses Netty as the inbuilt server to run reactive applications. If you are familiar with the Spring MVC programming style, you can easily work on webflux also.

Spring webflux uses project reactor as the reactive library. Reactor is a Reactive Streams library; therefore, all of its operators support non-blocking back pressure. It is developed in close collaboration with Spring.

Spring WebFlux heavily uses two publishers :

- **Mono**: Returns 0 or 1 element.
- **Flux**: Returns 0...N elements.

```
Mono<String> mono = Mono.just("Alex");  
Mono<String> mono = Mono.empty();
```

```
Flux<String> flux = Flux.just("A", "B", "C"); Flux<String> flux = Flux.fromArray(new  
String[]{"A", "B", "C"}); Flux<String> flux = Flux.fromIterable(Arrays.asList("A", "B",  
"C")); //To subscribe call method flux.subscribe();
```

Practical session 7-1

Do it yourself



Event Driven Architecture – Kafka – PS7-1

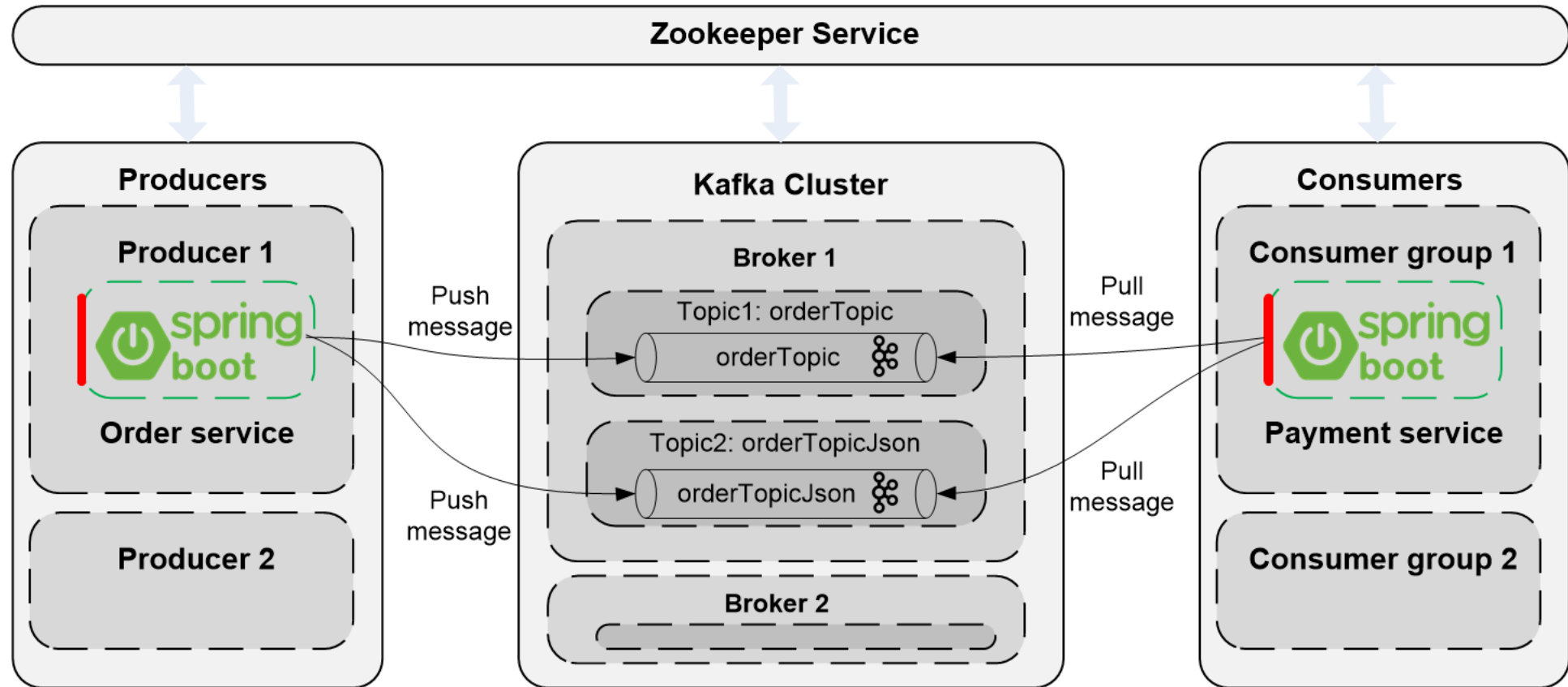
In this example, we have two microservices:

- **order-service** plays the message/event producer.
- **payment-service** plays the message/event **consumer**.

Starting Zookeeper and Kafka broker.

- 1- Clone the repo.
- 2- Run **Docker desktop**.
- 3- Create **docker-compose.yml**, and run it (\$ docker compose up -d).
- 4- Run the **services** and send/consume messages/objects.

Event Driven Architecture – Kafka – PS7-1



order-service

```
//KafkaTopicConfiguration.java

@Configuration
public class KafkaTopicConfiguration {

    @Bean
    public NewTopic OrderTopicCreation() {
        return TopicBuilder.name("orderTopic")
            .build();
    }

    @Bean
    public NewTopic OrderTopicCreationJson() {
        return TopicBuilder.name("orderTopicJson")
            .build();
    }
}
```

order-service

```
// // ProducerController.java
//
// ProducerController.java
//
// @RequestMapping("/api/kafka")
//
// public class ProducerController {
//
//     @Autowired
//     private ProducerService producerService;
//
//     @GetMapping("/publish")
//     public ResponseEntity<String> publish(@RequestParam("message") String message) {
//         producerService.sendMessageToOrderTopic(message);
//         return ResponseEntity.ok("Message sent to the order topic");
//     }
//
//     @PostMapping("/publish")
//     public ResponseEntity<String> publishjson(@RequestBody Order order) {
//         producerService.sendJsonToOrderTopic(order);
//         return ResponseEntity.ok("Json order object sent to the order topic");
//     }
// }
```

order-service

```
// // ProducerController.java
// // ProducerService.java
// @ // ProducerService.java
// p public class ProducerService {
//
//     private final KafkaTemplate<String, String> kafkaTemplate;
//
//     public void sendMessageToOrderTopic(String message){
//         log.info("Message send to order topic: {} ", message);
//         kafkaTemplate.send("orderTopic", message);
//     }
//
//     private final KafkaTemplate<String, Order> jsonKafkaTemplate;
//
//     public void sendJsonToOrderTopic(Order order){
//         log.info("Log message - Send order json object to order topic: {} ",
// order.toString());
//         jsonKafkaTemplate.send("orderTopicJson", order);
//     }
// }
// }
```

payment-service

```
//ConsumerService.java
```

```
@Service
```

```
public class ConsumerService {
```

```
@KafkaListener(topics = "orderTopic", groupId = "orderEventGroup")
```

```
    public void consumes(String message) {
```

```
        log.info("Log message - Message recieved from order topic: {} ", message);
```

```
    }
```

```
@KafkaListener(topics = "orderTopicJson", groupId = "orderEventGroup" )
```

```
public void consumes(Order order) {
```

```
    log.info("Log message recieved from order topic: {} ", order.toString());
```

```
}
```

```
}
```

application.properties

```
##### order service - Producer #####
spring.kafka.producer.bootstrap-servers: localhost:9092
spring.kafka.producer.key-serializer: org.apache.kafka.common.serialization.StringSerializer
#spring.kafka.producer.value-serializer: org.apache.kafka.common.serialization.StringSerializer

spring.kafka.producer.value-serializer:
org.springframework.kafka.support.serializer.JsonSerializer
spring.kafka.producer.properties.spring.json.type.mapping=order:com.esi.orderservice.model.Order
```

```
##### payment service - Consumer #####
spring.kafka.consumer.bootstrap-servers: localhost:9092
spring.kafka.consumer.group-id: orderEventGroup
spring.kafka.consumer.auto-offset-reset: earliest

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
#spring.kafka.consumer.value-deserializer:
org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-
deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.type.mapping=order:com.esi.paymentservice.model.Order
```

Practical session 7-2

Event Driven Architecture – Kafka – PS7-2

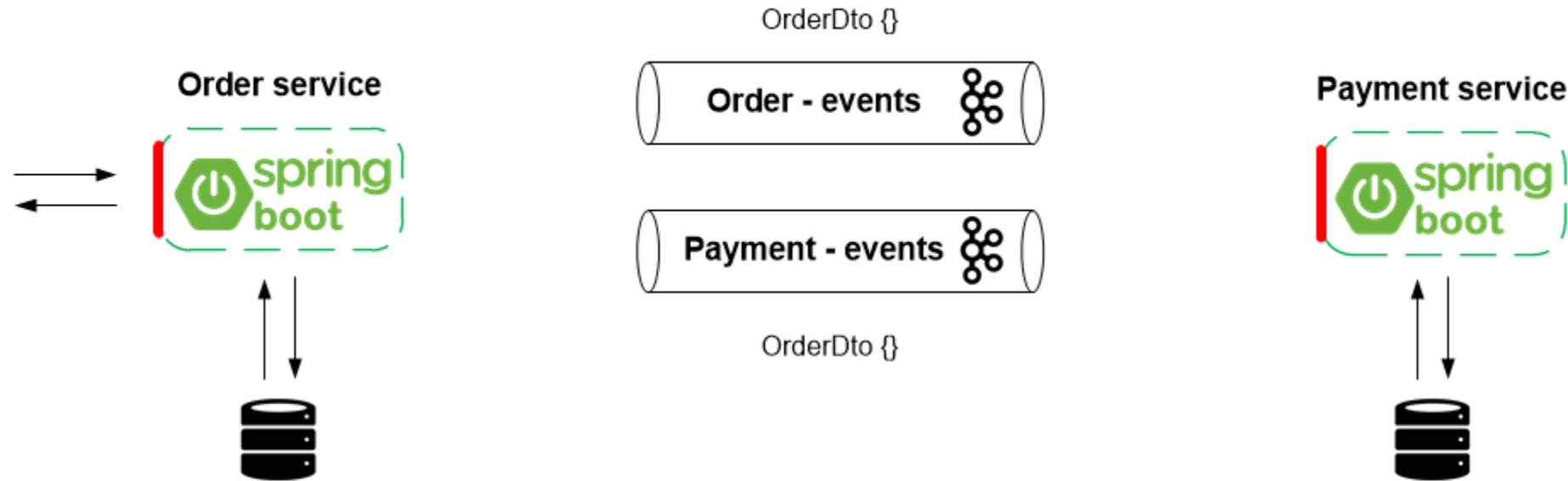


id	userId	productId	price	orderStatus	paymentStatus
..	..	..	..	..	..
..	..	..	..	..	..
..	..	..	..	..	..

In this example, we have two microservices:

- **order-service.**
- **payment-service.**

Event Driven Architecture – Kafka – PS7-2

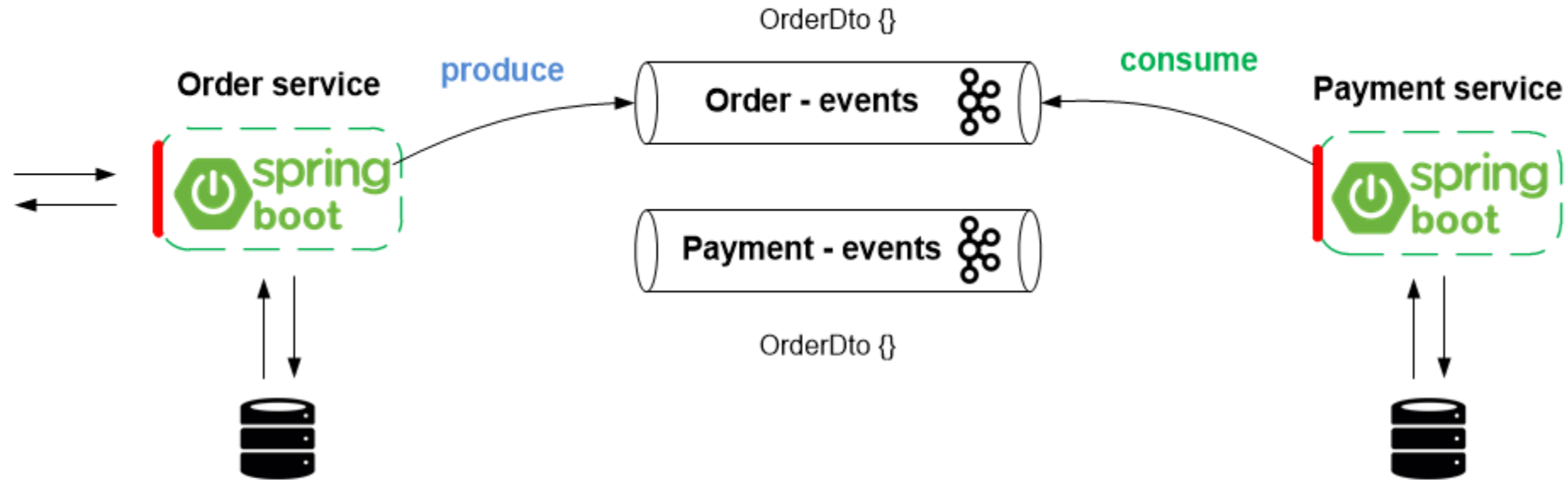


id	userId	productId	price	orderStatus	paymentStatus
..	..	..	..	..	..
..	..	..	..	..	..
..	..	..	..	..	..

In this example, we have two microservices:

- **order-service.**
- **payment-service.**

Event Driven Architecture – Kafka – PS7-2



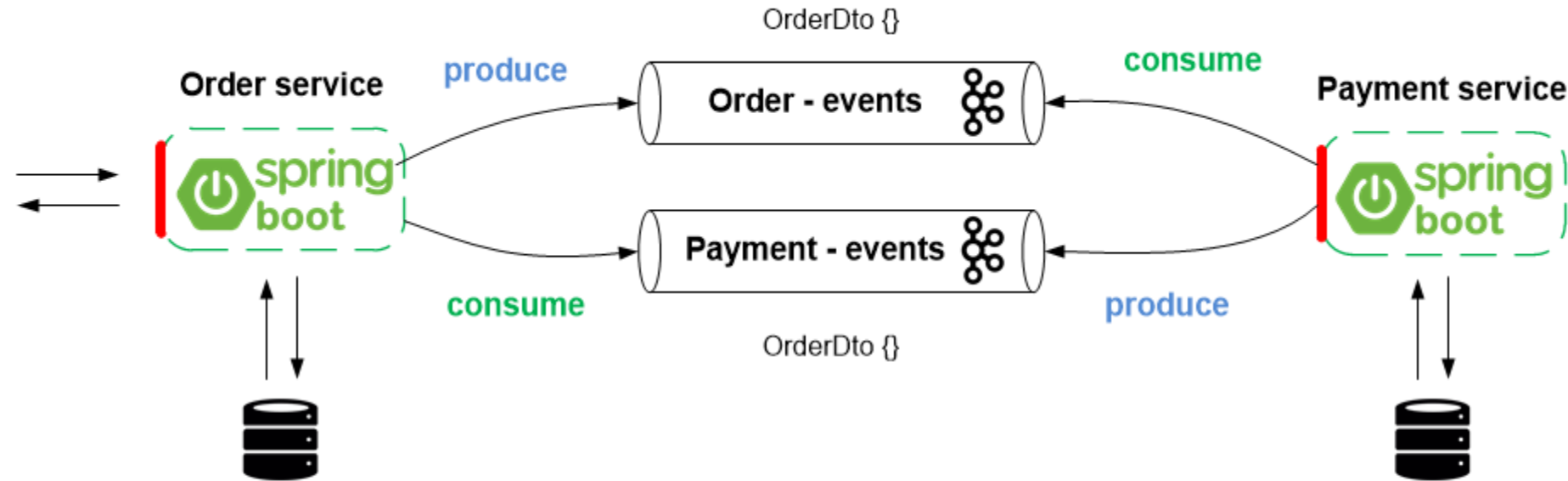
id	userId	productId	price	orderStatus	paymentStatus
..	..	..	..	..	..
..	..	..	..	..	..
..	..	..	..	..	..

userId	balance
..	..
..	..
..	..

In this example, we have two microservices:

- **order-service.**
- **payment-service.**

Event Driven Architecture – Kafka – PS7-2



id	userId	productId	price	orderStatus	paymentStatus
..	..	..	..	..	..
..	..	..	..	..	..
..	..	..	..	..	..

userId	balance
..	..
..	..
..	..

In this example, we have two microservices:

- **order-service.**
- **payment-service.**



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

